PROJECT REPORT

on

Health+

from

FabAppsLab Pvt.Ltd

Towards partial fulfillment of the requirements for the award of degree of

Bachelor of Computer Applications
(Data Science & Artificial Intelligence)
from

Babu Banarasi Das University Lucknow

Academic Session 2025 – 26

Department of Data Science & Artificial Intelligence
School of Computer Applications

I Floor, CA- BLOCK, BBD University, BBD City, Ayodhya Road, Lucknow (U. P.) INDIA 226028
PHONE: HEAD: 0522-6196242 **Dept. T&P Cell:** 9990462250

www.bbdu.ac.in

PROJECT REPORT

on

Health+

from

FabAppsLab Pvt.Ltd

Towards partial fulfillment of the requirements
for the award of degree of
Bachelor of Computer Applications
(Data Science & Artificial Intelligence)
from

Babu Banarasi Das University Lucknow

Submitted by
Khushi Kotak
University Roll No.
1230258219

Under Guidance of
Kartikey Dwivedi
Assistant professor

Academic Session 2025 – 26

Department of Data Science & Artificial Intelligence
School of Computer Applications

UNDER TAKING

This is to certify that Project Report entitled

Health+

being submitted by

Khushi Kotak

Towards the partial fulfillment of the requirements for the award of the degree of

Bachelor of Computer Applications
(Data Science & Artificial Intelligence)

from

Babu Banarasi Das University
Lucknow

Academic Year 2025-26 is a record of the student's own work carried out at

Fab Apps Lab Pvt.Ltd

and to the best of our knowledge the work reported herein does not form a part of any other thesis or work on the basis of which degree or award was conferred on an earlier occasion to this or any other candidate.

Authorized Signatory
Fab Apps Lab Pvt.Ltd

Students Signature:
Name: Khushi Kotak
Roll No.:1230258219

DECLARATION

I hereby declare that the report entitled "**Health+**", submitted to the **Department of Data Science & Artificial Intelligence, School of Computer Applications, Babu Banarasi Das University,** *BBD City, Ayodhya Road, Lucknow, Uttar Pradesh – 226028*, is submitted in partial fulfilment of the requirements for the award of the degree of **Bachelor of Computer Applications (Data Science & Artificial Intelligence).**

This report is the outcome of my own effort and has been completed under the guidance and supervision of **Project Guide Name, Designation, Department of Data Science & Artificial Intelligence, School of Computer Applications, Babu Banarasi Das University, Lucknow**. I also declare that the contents of this report have not been submitted, either in full or in part, to any other university or institution for the award of any degree or diploma.

Place: Lucknow (UP)
Date: _____
Student Name: _____
University Roll No.: _____
Signature of the Student: _____

\

ACKNOWLEDGEMENT

The completion of this project marks not only an academic milestone but also a deeply personal journey filled with learning, challenges, and growth. I would like to express my heartfelt gratitude to all those who supported and guided me throughout this journey. The successful completion of this work would not have been possible without the encouragement and assistance of many individuals.

First and foremost, I would like to extend my profound gratitude to **Dr. Reena Srivastava,** Honorable **Dean**, School of Computer Applications, Babu Banarasi Das University for her continuous support, motivation, and encouragement. Her visionary leadership and guidance have always been a source of inspiration for us.

I am also deeply thankful to **Dr. Manish Kumar**, **Head of the Department**, Department of Cyber Security & Forensics for his constant support and valuable guidance, which played a vital role in helping me stay focused and complete this project successfully.

I would like to express my sincere gratitude to **Mr. Sri Nivash Singh, T&P Coordinator**, for his support, cooperation, and coordination throughout the project.

I extend my deep appreciation to my project guide, **Kartikeya Dwivedi**, **Assistant Professor,** for his unwavering support, expert guidance, patience, and insightful suggestions. His continuous encouragement and feedback greatly contributed to the successful completion of this work and enhanced my understanding.

Last but not the least, I sincerely thank my **Family** and **Friends,** and mentors for their constant support, encouragement, and motivation. Their belief in me has been a strong pillar behind my consistent efforts.

Table of Contents

1.INTRODUCTION OF THE PROJECT

1.1 Overview

Health+ Diet Planner is a comprehensive web-based health management platform designed to help users take control of their nutrition, fitness goals, and overall well-being. The application provides a suite of integrated tools including personalized calorie calculations, daily meal tracking, AI-powered diet plan generation, automated grocery list creation, water intake monitoring, and medication reminder services via SMS.

The platform combines modern web technologies with intelligent algorithms to deliver a seamless user experience. By leveraging the Mifflin-St Jeor equation for BMR calculation and activity-based TDEE estimation, the system provides scientifically-grounded nutritional guidance tailored to individual users.

1.2 Background

In today's fast-paced world, maintaining a healthy diet and lifestyle has become increasingly challenging. Many individuals struggle with understanding their nutritional needs, tracking their daily food intake, and planning balanced meals. The Health+ Diet Planner was developed to address these challenges by providing an all-in-one platform that simplifies health management.

The platform integrates eight key modules: Smart Calorie System for dynamic BMR and TDEE calculation, User Profile Management for personalized health tracking, a Daily Meal Tracker with 7-day history, an interactive Dashboard with visual analytics, an AI-powered Meal Recommendation Engine, an automated Grocery List Generator, a Water Intake Tracker, and a Medication Reminder System powered by Twilio SMS integration.

1.3 Key Features

- Smart Calorie System: Calculates BMR using the Mifflin-St Jeor equation, computes TDEE based on activity level, and adjusts calorie targets based on fitness goals (weight loss, muscle gain, maintenance)
- User Profile Management: Stores personal health data including weight, height, age, gender, activity level, goal, diet type, and allergies using browser localStorage for complete data privacy
- Daily Meal Tracker: Enables users to log meals with nutritional details (calories, protein, carbs, fat), displays consumed vs remaining calories, and maintains a 7-day meal history with automatic pruning
- Interactive Dashboard: Provides stat cards showing daily calorie target, consumed calories, remaining budget, and protein intake, along with Chart.js-powered weekly trend bar charts and macro distribution doughnut charts
- AI Diet Plan Generation: Integrates with Google Gemini 1.5 Flash API to generate personalized meal plans based on user biometrics, dietary preferences, and allergies
- Meal Recommendation Engine: Contains a database of 50+ meals filtered by remaining calorie budget and protein requirements, sorted by nutritional relevance
- Grocery List Generator: Automatically generates categorized grocery lists from AI-generated diet plans, with support for manual item addition and category-based organization

- Water Intake Tracker: Simple increment/decrement counter with daily goal tracking (default 8 glasses) and visual progress bar
- Medication Reminder: Allows users to schedule SMS reminders for medications using Twilio

  API integration with phone number storage

1.4 Objectives

- To provide personalized calorie targets based on user biometrics and fitness goals
- To enable daily meal logging with real-time nutritional progress tracking
- To generate AI-powered diet plans using Google Gemini API integration
- To automate grocery list creation from meal plans, saving user time
- To provide visual analytics through interactive charts and dashboards
- To implement a medication reminder system via SMS notifications
- To ensure data privacy by using client-side localStorage for all personal data
- To deliver a mobile-responsive, modern UI/UX experience

1.5 Technologies Used

Technology	Version	Purpose
HTML5	5	Semantic page structure and accessibility
CSS3	3	Styling with custom properties, Flexbox, Grid, media queries
JavaScript (ES6+)	ES2020+	Client-side business logic, modular architecture
Node.js	18+	Server-side runtime environment
Express.js	4.21	REST API framework for backend routes
Chart.js	4.4.7	Data visualization (bar charts, doughnut charts)
Google Gemini API	1.5 Flash	AI-powered diet plan generation
Twilio SDK	4.21	SMS medication reminder service
localStorage	Web API	Client-side data persistence
dotenv	17.2	Environment variable management

2. NEED OF IDENTIFICATION

2.1 Problem Domain Analysis

The health and nutrition management domain faces several critical challenges that affect millions of users worldwide. Most existing solutions provide fragmented functionality, requiring users to switch between multiple applications for different health management tasks. This fragmentation leads to poor user engagement, inconsistent data tracking, and ultimately, failure to achieve health goals.

The key problems identified in the domain include the lack of personalized calorie recommendations based on individual biometrics, absence of integrated meal tracking with nutritional analytics, manual and time-consuming grocery planning processes, and the disconnect between diet planning and medication management.

2.2 Need for the Platform

The Health+ Diet Planner addresses the fundamental need for a unified health management platform that combines multiple health-related functionalities into a single, cohesive application. The platform eliminates the need for users to maintain separate applications for calorie counting, meal planning, grocery management, and medication reminders.

The system uses scientifically validated formulas (Mifflin-St Jeor equation) for calorie calculations, ensuring accuracy and reliability. By integrating AI-powered diet plan generation through Google Gemini, the platform provides intelligent, personalized meal recommendations that adapt to individual dietary preferences and restrictions.

2.3 Target Audience

- Health-conscious individuals seeking to manage their daily nutrition and calorie intake
- Fitness enthusiasts who need accurate BMR/TDEE calculations and macro tracking
- People with specific dietary requirements (vegetarian, vegan, omnivore) who need customized meal plans
- Individuals managing medications who need reliable SMS-based reminders
- Users who prefer privacy-first solutions with local data storage rather than cloud-based services
- Anyone looking for an integrated platform that combines diet planning, tracking, and grocery management

3. PROBLEM STATEMENT

3.1 Statement of the Problem

Design and develop a comprehensive web-based health management platform that provides personalized nutrition guidance through dynamic calorie calculations, AI-powered diet plan generation, daily meal tracking with nutritional analytics, automated grocery list creation, water intake monitoring, and medication reminder services. The system must deliver accurate, sciencebased nutritional recommendations while maintaining user data privacy through client-side storage, and must provide an intuitive, mobile-responsive user interface with visual analytics.

3.2 Existing System Limitations

Current health management solutions in the market suffer from several limitations that the Health+ Diet Planner aims to address:

- Static Calorie Goals: Most applications provide fixed calorie targets without considering individual biometrics such as BMR, TDEE, activity level, and fitness goals
- Fragmented Tooling: Users must switch between separate applications for calorie tracking, meal planning, grocery management, and medication reminders
- Privacy Concerns: Many platforms require cloud accounts and store sensitive health data on remote servers, raising privacy and data security concerns
- Poor Mobile Experience: Several desktop-first applications lack responsive design, making them difficult to use on mobile devices
- No Integrated Analytics: Basic tracking tools fail to provide meaningful visual analytics such as weekly trends, macro breakdowns, and progress charts
- Manual Grocery Planning: Users must manually create grocery lists from their meal plans, which is time-consuming and error-prone

3.3 Proposed System

The Health+ Diet Planner addresses all identified limitations through an integrated, privacy-first approach:

Dynamic Calorie System: Implements BMR calculation using the Mifflin-St Jeor equation (10*weight + 6.25*height - 5*age +/- gender factor), TDEE computation with five activity multipliers, and goal-based calorie adjustment (weight loss: -500 kcal, muscle gain: +300 kcal, maintenance: +0 kcal)

- Unified Platform: Combines eight modules (Profile, Dashboard, Tracker, Diet Planner, Grocery, Water, Recommendations, Reminders) into a single application with shared navigation and data flow
- Privacy-First Architecture: All personal data (profile, meals, water intake, grocery lists) stored exclusively in browser localStorage using JSON serialization, never transmitted to external servers

- Mobile-Responsive Design: CSS custom properties, Flexbox/Grid layouts, and media queries ensure optimal experience across all device sizes
- Visual Analytics: Chart.js integration provides interactive bar charts (weekly calorie trends) and doughnut charts (macro distribution) on the dashboard
- Automated Grocery Generation: Parses AI-generated diet plans to extract ingredients, categorizes them (Protein, Dairy, Grains, Produce, Pantry), and generates organized shopping lists

4. SYSTEM ANALYSIS

4.1 Need of Identification

A thorough analysis of the health management domain was conducted to identify the core needs that the Health+ Diet Planner must address. The following table summarizes the identified needs categorized by functional area:

Category	Need	Priority
Personalization	Dynamic calorie calculation based on user biometrics	High
Personalization	Goal-based nutritional adjustments	High
Tracking	Daily meal logging with nutritional breakdown	High
Tracking	Water intake monitoring with goal tracking	Medium
Analytics	Visual dashboard with charts and statistics	High
Analytics	Weekly trend analysis and macro distribution	Medium
Planning	AI-powered diet plan generation	High
Planning	Automated grocery list creation from meal plans	Medium
Communication	SMS-based medication reminders	Medium
UI/UX	Mobile-responsive modern interface	High
Privacy	Client-side data storage without cloud dependency	High

4.2 Preliminary Investigation

The preliminary investigation involved a comprehensive assessment of the project requirements, available technologies, and development approach. The investigation covered the following areas:

- Technology Assessment: Evaluated Node.js/Express for backend, vanilla JavaScript for frontend logic, Chart.js for visualization, and localStorage for client-side persistence

  API Feasibility: Confirmed Google Gemini 1.5 Flash API availability for diet plan generation and Twilio SMS API for medication reminders

- User Research: Identified target audience needs through analysis of existing health applications and their limitations
- Architecture Decision: Selected a modular frontend architecture with separate JavaScript modules (calories.js, storage.js, recommendations.js, ui.js) for maintainability

4.3 Feasibility Study

Technical Feasibility

The project is technically feasible using widely-available web technologies. HTML5, CSS3, and JavaScript are universally supported across browsers. Node.js and Express.js provide a robust backend framework. The Google Gemini API and Twilio API are well-documented with stable SDKs. localStorage provides up to 5-10 MB of client-side storage, sufficient for user profiles, meal logs, and grocery lists.

Economic Feasibility

The project demonstrates strong economic feasibility with minimal infrastructure costs:

- Development Tools: All development tools (VS Code, Node.js, Git) are open-source and free
- Hosting: Static files can be hosted on free-tier platforms; backend requires minimal compute resources
- APIs: Google Gemini API provides a generous free tier; Twilio offers trial credits for SMS testing
- No Database Costs: localStorage eliminates the need for database hosting and management

Operational Feasibility

The system is operationally feasible as it requires no installation beyond a modern web browser. Users can access the application through any device with internet connectivity. The intuitive UI design minimizes the learning curve, and the modular architecture enables easy maintenance and feature additions.

Time Feasibility

The project was planned and executed within an 18-week timeline, with clear milestones for each development phase. The use of modern web technologies and incremental development methodology enabled efficient delivery of all planned features within the scheduled timeframe.

4.4 Project Planning

The project was organized into well-defined work packages, each with specific deliverables and timelines:

WP#	Work Package	Duration	Deliverable
WP-1	Requirements Analysis	Week 1-3	SRS Document
WP-2	Feasibility Study	Week 2-4	Feasibility Report

WP-3	UI/UX Design	Week 3-6	Wireframes & Design System
WP-4	Profile Module Development	Week 4-7	User Profile Page
WP-5	Calorie Engine Development	Week 5-8	BMR/TDEE Calculator
WP-6	Meal Tracker Development	Week 6-9	Tracking Interface
WP-7	Dashboard Development	Week 7-10	Dashboard with Charts
WP-8	Recommendation Engine	Week 8-10	Meal Suggestions
WP-9	Grocery Module	Week 8-11	Grocery List Feature
WP-10	API Integration	Week 9-12	Gemini & Twilio
WP-11	Testing & QA	Week 10-15	Test Reports
WP-12	Documentation & Deploy	Week 14-18	Final Report

4.5 Project Scheduling

4.5.1 PERT Chart Analysis

The Program Evaluation and Review Technique (PERT) was applied to estimate task durations and identify the critical path through the project network. Each task was analyzed with optimistic (O), most likely (M), and pessimistic (P) time estimates:

Task	O (weeks)	M (weeks)	P (weeks)	Expected
Feasibility Study	1	2	4	2.2
UI/UX Design	2	3	5	3.2
Profile Module	2	3	4	3.0
Calorie Engine	2	3	5	3.2
Meal Tracker	2	3	5	3.2
Dashboard Module	2	3	4	3.0
API Integration	2	3	5	3.2
Testing & QA	3	4	7	4.3
Documentation	2	3	5	3.2

Requirements Analysis	2	3	4	3.0

Critical Path: Requirements Analysis -> UI/UX Design -> Calorie Engine -> Meal Tracker -> Dashboard -> Testing -> Documentation

Total Critical Path Duration: 22.8 weeks (estimated)

4.7 Gantt Chart

The following Gantt chart provides a visual representation of the project schedule, showing task durations, dependencies, and milestones:

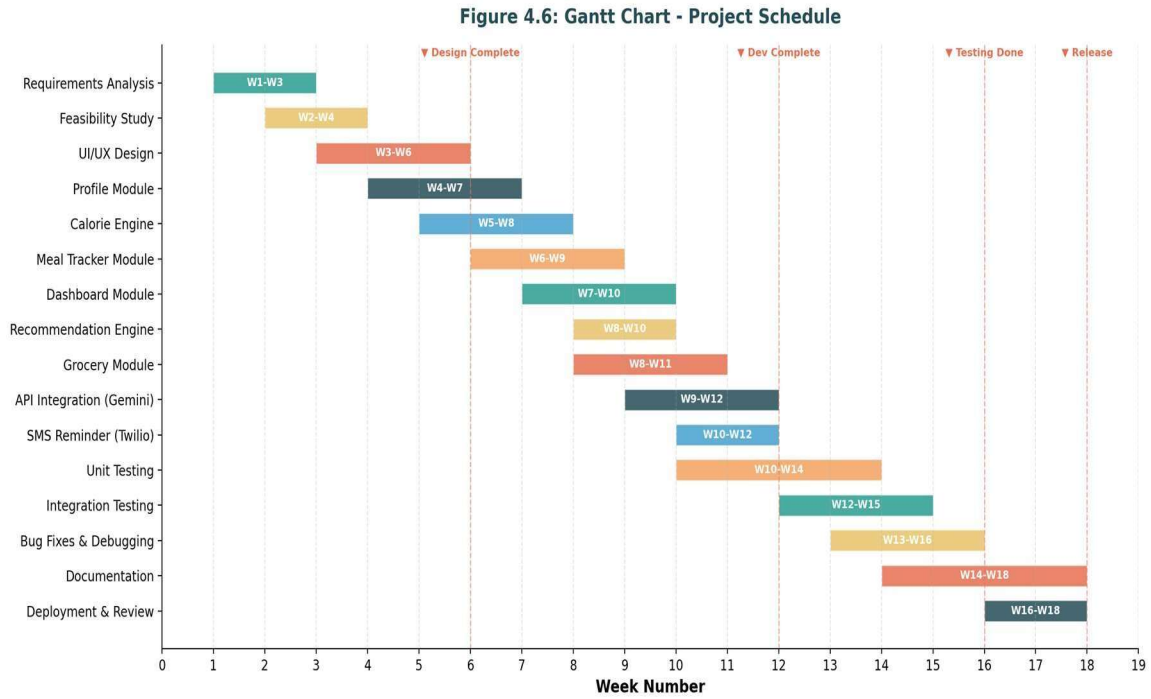

Figure 4.6: Gantt Chart - Project Schedule

4.8 Software Requirement Specifications (SRS)

Functional Requirements

The Health+ Diet Planner system shall provide the following functional capabilities:

- FR-01: The system shall allow users to create and edit personal health profiles with fields for name, age, gender, height (cm), weight (kg), activity level, fitness goal, diet type, and allergies
- FR-02: The system shall calculate Basal Metabolic Rate (BMR) using the Mifflin-St Jeor equation based on the user's biometric data
- FR-03: The system shall compute Total Daily Energy Expenditure (TDEE) by multiplying BMR with the appropriate activity multiplier (Sedentary: 1.2, Light: 1.375, Moderate: 1.55, Active: 1.725, Very Active: 1.9)
- FR-04: The system shall adjust target calories based on the user's fitness goal: Weight Loss (500 kcal), Muscle Gain (+300 kcal), Maintenance (no adjustment), with a minimum floor of 1200 kcal/day
- FR-05: The system shall calculate recommended macro splits (protein, carbs, fat in grams) based on the target calories and fitness goal
- FR-06: The system shall provide a dashboard displaying daily calorie target, consumed calories, remaining budget, and protein intake as stat cards

- FR-07: The system shall render interactive charts including a weekly calorie trend bar chart and a macro distribution doughnut chart using Chart.js
- FR-08: The system shall allow users to log meals with fields for meal name, type (breakfast/lunch/dinner/snack), calories, protein, carbs, and fat
- FR-09: The system shall display consumed vs remaining calories with visual progress indicators on the Tracker page
- FR-10: The system shall maintain a 7-day meal history and automatically prune entries older than 7 days
- FR-11: The system shall recommend meals from a database of 50+ options, filtered by remaining calorie budget and protein requirements
- FR-12: The system shall generate personalized diet plans using the Google Gemini 1.5 Flash API based on user biometrics and dietary preferences
- FR-13: The system shall automatically generate categorized grocery lists from AI-generated diet plans, organizing items into categories (Protein, Dairy, Grains, Produce, Pantry)
- FR-14: The system shall allow manual addition and removal of grocery items with category selection
- FR-15: The system shall provide a water intake tracker with increment/decrement functionality and daily goal tracking (default: 8 glasses)
- FR-16: The system shall allow users to set medication reminders with phone number, medication name, and time
- FR-17: The system shall send SMS reminders using the Twilio API to the user's registered phone number
- FR-18: The system shall persist all user data in browser localStorage using JSON serialization
- FR-19: The system shall propagate profile data across all pages (Dashboard, Tracker, Grocery) for consistent calorie targets
- FR-20: The system shall provide a responsive navigation bar with links to all modules

Non-Functional Requirements

- NFR-01: Performance - All pages shall load within 2 seconds on standard broadband connections
- NFR-02: Responsiveness - The UI shall adapt to screen sizes from 320px (mobile) to 2560px (4K desktop)
- NFR-03: Browser Compatibility - The application shall function correctly on Chrome 90+, Firefox 88+, Safari 14+, and Edge 90+
- NFR-04: Data Privacy - No personal health data shall be transmitted to external servers except when explicitly invoking API features (diet generation, SMS)
- NFR-05: Security - API keys shall be stored server-side in .env files, never exposed in clientside code
- NFR-06: Usability - The application shall be operable without any prior training or tutorial
- NFR-07: Maintainability - Code shall follow modular architecture with separate JS modules for each concern

- NFR-08: Availability - The frontend shall remain functional offline for all localStorage-based features
- NFR-09: Scalability - The architecture shall support addition of new modules without modifying existing ones
- NFR-10: Accessibility - All interactive elements shall be keyboard-navigable with appropriate ARIA attributes

4.9 Software Engineering Paradigm Applied

The Health+ Diet Planner was developed using the Incremental Development Model, which is a combination of the iterative and waterfall approaches. In this paradigm, the system is designed, implemented, and tested incrementally until the product is complete.

The incremental approach was chosen for the following reasons:

- Each module (Profile, Dashboard, Tracker, etc.) could be developed and tested independently before integration
- Stakeholder feedback could be incorporated after each increment, reducing the risk of building unwanted features
- Core functionality (calorie calculations, data storage) was delivered first, with advanced features (AI diet plans, SMS reminders) added in subsequent increments
- Testing was performed after each increment, ensuring early detection of defects and maintaining code quality throughout development

4.10 Data Flow Diagrams

Data Flow Diagrams (DFDs) are used to represent the flow of data through the Health+ system at different levels of abstraction.

Context Diagram (Level 0 DFD)

The Context Diagram represents the entire Health+ Diet Planner system as a single process interacting with external entities:

Figure 4.1: Context Diagram (Level 0 DFD)

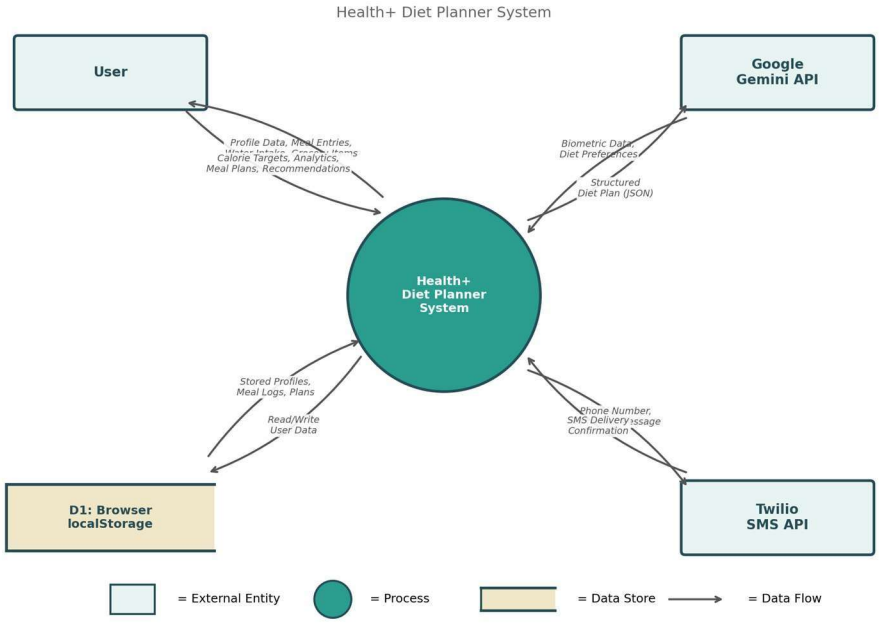

Figure 4.1: Context Diagram (Level 0 DFD)

Level 1 DFD

The Level 1 DFD decomposes the system into seven major processes, showing detailed data flows between processes, external entities, and data stores:

Figure 4.2: Level 1 Data Flow Diagram

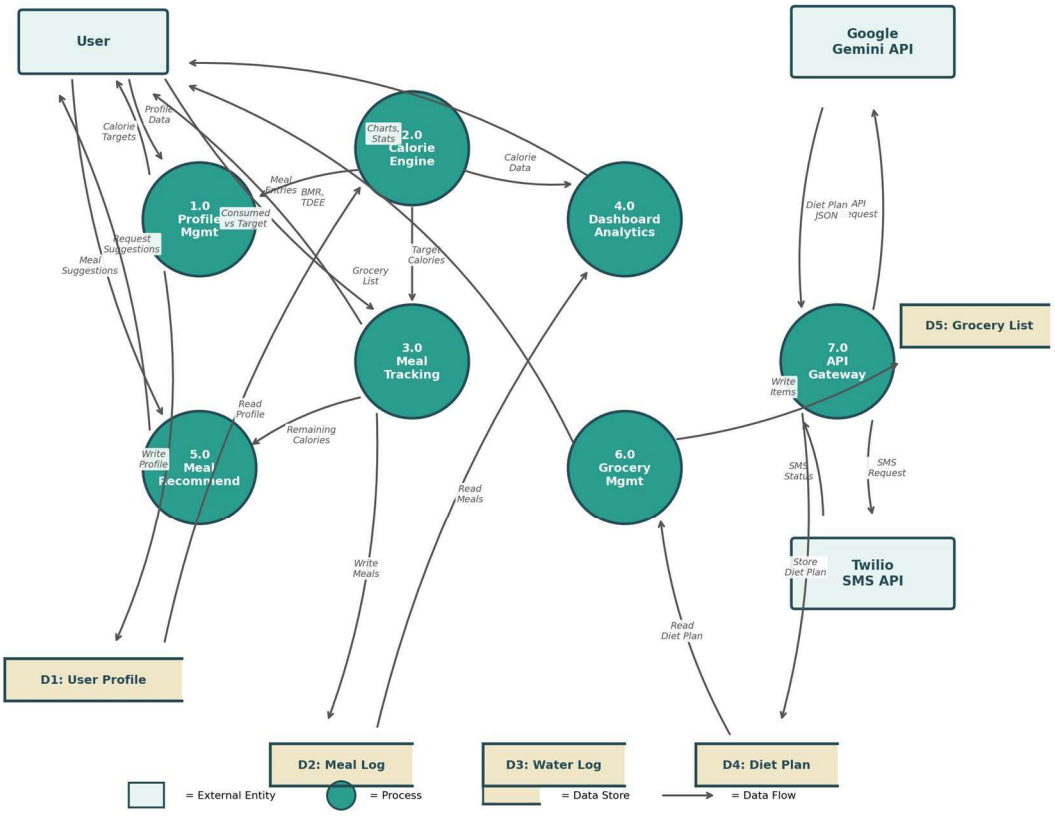

Figure 4.2: Level 1 Data Flow Diagram

Use Case Diagram

The Use Case Diagram illustrates the interactions between the User actor and the system's functional capabilities, along with external API actors (Gemini and Twilio):

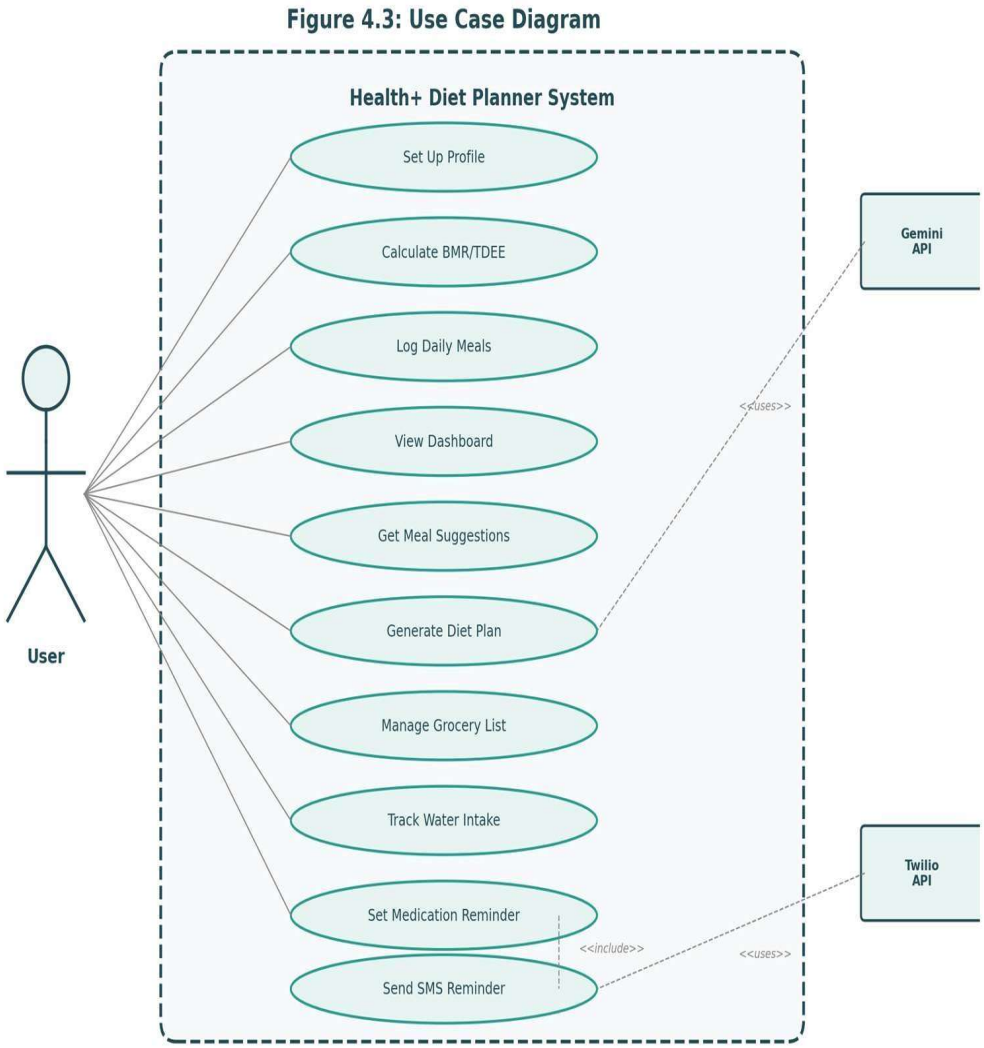

Figure 4.3: Use Case Diagram

Activity Diagram

The Activity Diagram depicts the workflow for the meal logging process, showing the sequence of activities, decision points, and error handling:

Figure 4.4: Activity Diagram - Meal Logging

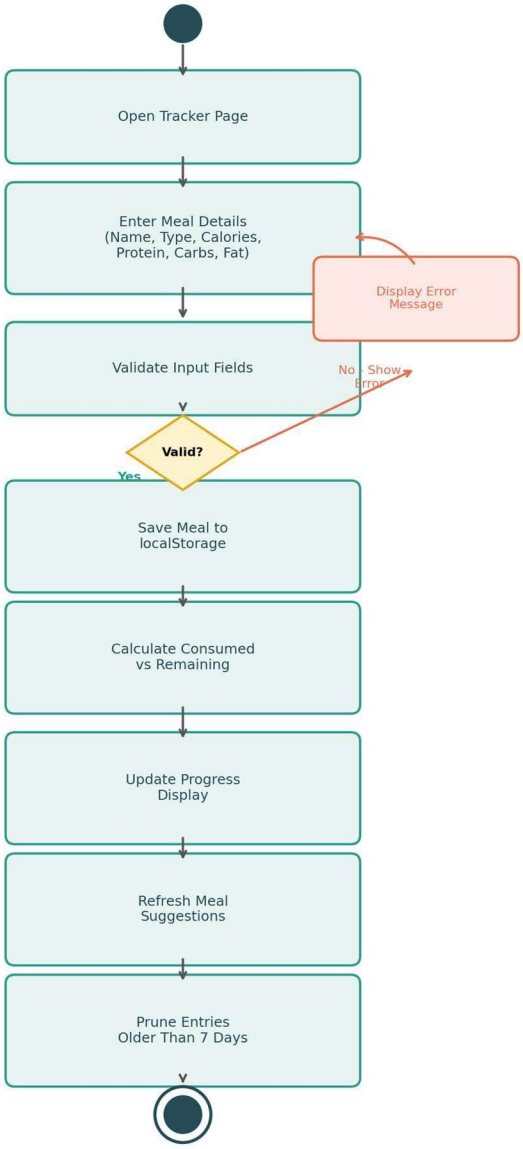

Figure 4.4: Activity Diagram - Meal Logging Flow

Sequence Diagram

The Sequence Diagram shows the interaction between objects during the meal logging and dashboard update workflow:

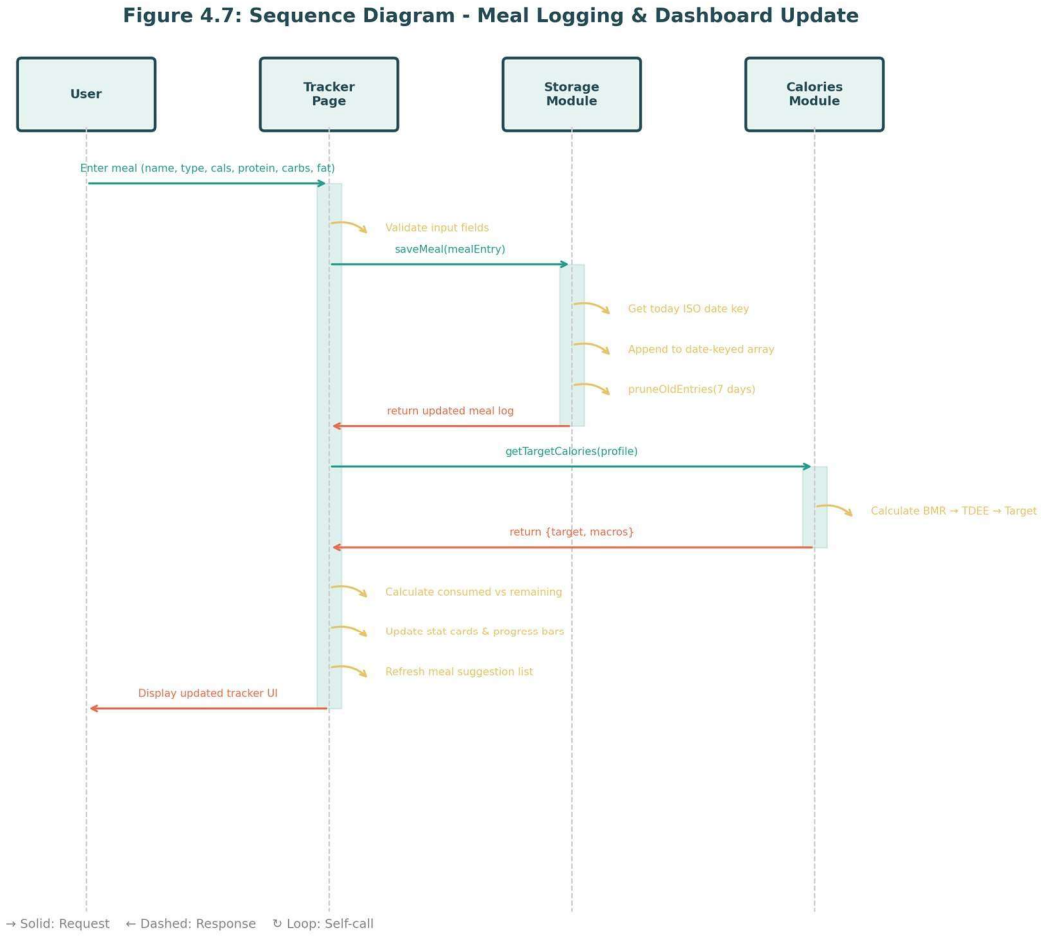

Figure 4.7: Sequence Diagram - Meal Logging & Dashboard Update

Component Diagram

The Component Diagram shows the structural organization of the Health+ system, depicting all software components and their dependencies:

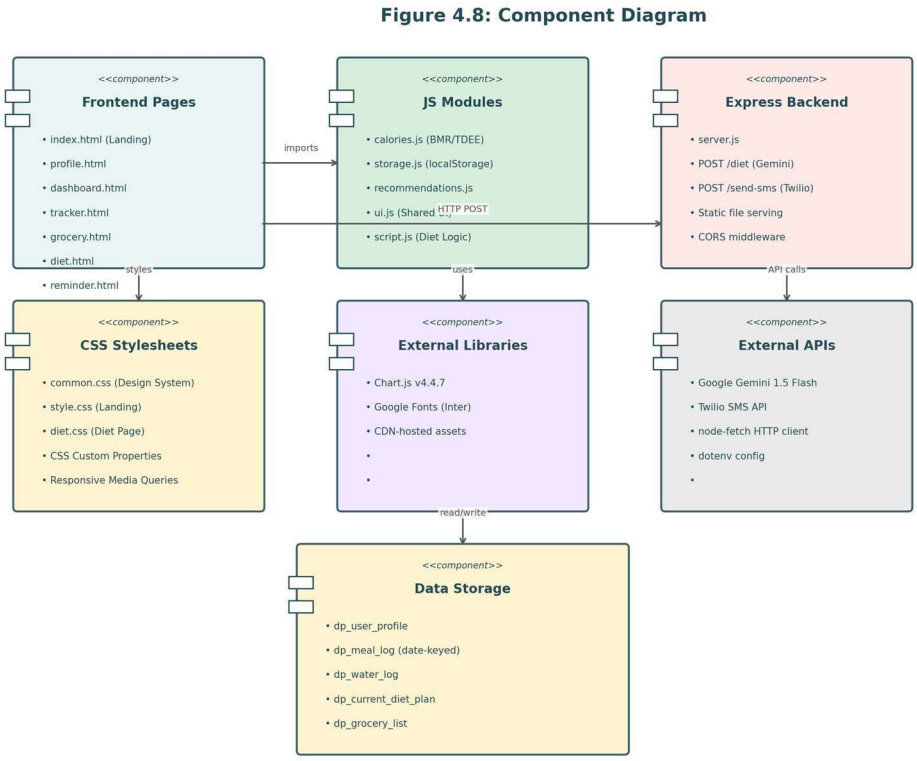

Figure 4.8: Component Diagram

UML Class/Module Diagram

The UML Class Diagram represents the major classes/modules in the system with their attributes, methods, and relationships:

Figure 4.9: UML Class/Module Diagram

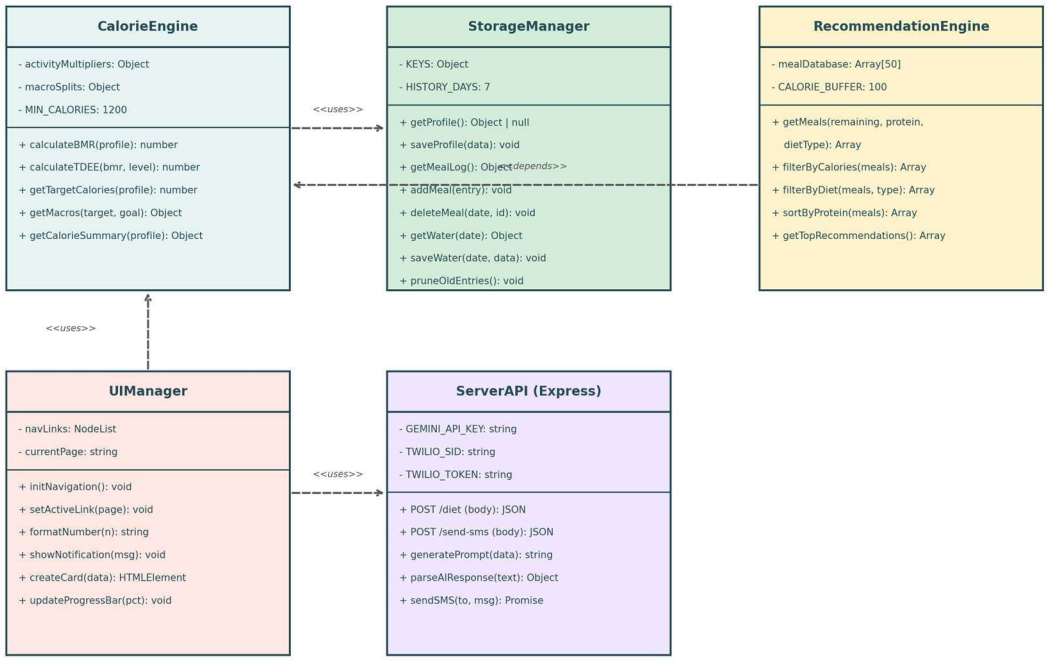

Figure 4.9: UML Class/Module Diagram

5. HIGH LEVEL DESIGN

5.1 System Architecture

The Health+ Diet Planner follows a layered architecture pattern with clear separation of concerns across four primary layers:

- Presentation Layer: HTML5 pages with responsive CSS, Chart.js visualizations, and interactive forms
- Business Logic Layer: Modular JavaScript files (calories.js, storage.js, recommendations.js, ui.js) implementing core algorithms and data processing
- Data Storage Layer: Browser localStorage providing JSON-serialized persistence for user profiles, meal logs, water intake records, diet plans, and grocery lists
- API Integration Layer: Express.js backend proxying requests to Google Gemini API for diet plan generation and Twilio API for SMS reminders

5.2 Architecture Decisions

Decision	Choice	Rationale
Frontend Framework	Vanilla JavaScript	No build tooling needed, direct browser execution, smaller bundle size
State Management	localStorage + URL hash	Privacy-first, no backend dependency for core features, persistent across sessions
Visualization	Chart.js v4.4.7	Lightweight library, supports bar and doughnut charts, responsive by default
Backend Runtime	Node.js + Express	JavaScript consistency across stack, async I/O for API proxying
AI Integration	Google Gemini 1.5 Flash	Generous free tier, structured JSON output, fast response times
SMS Service	Twilio	Reliable delivery, simple REST API, trial credits for development
CSS Architecture	Custom Properties (Design Tokens)	Consistent theming, easy modification, no preprocessor required
Module System	ES Modules (type=module)	Native browser support, clean import/export syntax

5.3 Page Navigation Flow

The application uses a shared navigation bar component rendered by ui.js across all pages. Navigation follows a hub-and-spoke pattern with the Landing Page (index.html) as the central hub and six functional pages as spokes:

- Landing Page (index.html) -> Profile Page (profile.html): User sets up biometric data
- Profile Page -> Dashboard (dashboard.html): Dashboard reads profile from localStorage to display stats and charts
- Dashboard -> Tracker (tracker.html): User navigates to log daily meals
- Dashboard -> Grocery (grocery.html): User manages grocery lists
- Navigation Bar -> Diet Plan (diet.html): User generates AI diet plans
- Navigation Bar -> Reminders (reminder.html): User sets medication reminders

5.4 Data Flow Architecture

The system implements a unidirectional data flow pattern where user input flows through validation, processing, and storage layers before being rendered in the UI:

- Input Flow: User form data -> Validation (type checking, bounds) -> Processing (BMR/TDEE calculation) -> Storage (localStorage) -> UI Update
- Cross-Page Flow: Profile data saved in localStorage is read by Dashboard, Tracker, and Grocery pages to display personalized information
- API Flow: Frontend -> Express Backend -> External API (Gemini/Twilio) -> Backend Response Parsing -> Frontend Display

6. LOW LEVEL DESIGN

6.1 Data Dictionary

The following data structures define the core entities stored in the Health+ system:

Structure	Key	Fields	Storage Key
UserProfile	dp_user_profile	name, age, gender, height, weight, activityLevel, goal, dietType, allergies	localStorage
MealLog	dp_meal_log	{ date: [{ id, name, type, calories, protein, carbs, fat, timestamp }] }	localStorage
WaterLog	dp_water_log	{ date: { glasses, goal, lastUpdated } }	localStorage
DietPlan	dp_current_diet_plan	{ meals[], snacks[], totalCalories, notes }	localStorage
GroceryList	dp_grocery_list	[{ id, name, category, quantity, checked }]	localStorage

6.2 Module Design

calories.js - Calorie Calculation Engine

- calculateBMR(profile): Implements Mifflin-St Jeor equation - Male: 10*w + 6.25*h - 5*a + 5, Female: 10*w + 6.25*h - 5*a - 161
- calculateTDEE(bmr, activityLevel): Multiplies BMR by activity factor (sedentary=1.2, light=1.375, moderate=1.55, active=1.725, veryActive=1.9)
- getTargetCalories(profile): Adjusts TDEE by goal (weightLoss=-500, muscleGain=+300, maintenance=0), enforces 1200 kcal minimum
- getMacroRecommendation(targetCalories, goal): Returns protein/carbs/fat grams based on goal-specific percentage splits
- getCalorieSummary(profile): Aggregates BMR, TDEE, target, and macros into a single summary object

storage.js - Data Persistence Manager

- getProfile() / saveProfile(data): Read/write user profile JSON to localStorage key 'dp_user_profile'
- getMealLog() / addMeal(entry): Date-keyed meal log management with ISO date keys (YYYY-MM-DD)
- deleteMeal(date, id): Remove specific meal entry by date and ID
- getWaterIntake(date) / saveWaterIntake(date, data): Daily water tracking read/write

- pruneOldEntries(): Removes meal log entries older than 7 days to prevent unbounded storage growth

recommendations.js - Meal Suggestion Engine

- Contains database of 50+ meals with nutritional data (name, calories, protein, carbs, fat, type, dietType)
- getRecommendations(remainingCalories, proteinTarget, dietType): Filters meals by calorie budget, diet compatibility, and sorts by protein content

ui.js - Shared UI Components

- initNavigation(): Sets up responsive navigation bar with active page highlighting
- formatNumber(n): Formats numeric values with locale-appropriate separators
- showNotification(message, type): Displays toast notifications for user feedback

7. SYSTEM DESIGN

7.1 Modularity and System Decomposition

The Health+ Diet Planner is decomposed into seven functional modules, each with a welldefined responsibility and interface:

- Profile Module (profile.html): Handles user input collection, validation, BMR/TDEE calculation display, and profile persistence. Imports calories.js and storage.js.
- Dashboard Module (dashboard.html): Reads profile and meal data to render stat cards, Chart.js charts (weekly trends, macro doughnut), water tracker, and meal suggestions. Imports all four JS modules.
- Tracker Module (tracker.html): Manages meal logging form, consumed/remaining display, meal list rendering, and history management. Imports calories.js, storage.js, and recommendations.js.
- Diet Planner Module (diet.html): Collects biometric data, sends POST request to /diet endpoint, parses AI response, and displays structured meal plan. Contains inline script.js logic.
- Grocery Module (grocery.html): Auto-generates grocery items from saved diet plan, supports manual add/remove, category filtering, and check/uncheck functionality. Imports storage.js.
- Reminder Module (reminder.html): Schedules medication reminders, sends POST to /sendsms endpoint, stores phone number locally, manages reminder timers. Contains inline script.
- Landing Module (index.html): Entry point with feature cards, navigation links, and hero section. Minimal JS for modal interactions.

7.2 *Data Integrity and Constraints*

The system enforces data integrity through multiple validation layers:

- Input Validation: All form inputs use HTML5 validation attributes (required, type=number, min, max) supplemented by JavaScript validation before storage
- Type Casting: All numeric inputs are cast using Number() to prevent string concatenation errors in calculations
- Bounds Checking: Age (1-120), Height (50-300cm), Weight (20-500kg) are validated with min/max constraints
- Date Isolation: Meal logs use ISO date keys (YYYY-MM-DD) to prevent cross-day data mixing
- Minimum Safety Floor: Target calories are clamped to a minimum of 1200 kcal/day regardless of goal adjustments
- Automatic Pruning: Meal log entries older than 7 days are automatically removed to prevent unbounded localStorage growth

7.3 Database Design (localStorage Schema)

The application uses browser localStorage as its primary data store. Data is organized into five independent JSON-serialized keys:

Key	Structure	Max Size	Pruning Policy
dp_user_profile	Single JSON object	~1 KB	Overwritten on save
dp_meal_log	Date-keyed object { YYYY-MM-DD: [...meals] }	~50 KB	Auto-prune > 7 days
dp_water_log	Date-keyed object { YYYY-MM-DD: {glasses, goal} }	~5 KB	One entry per day
dp_current_diet_plan	Single JSON object with meals array	~10 KB	Overwritten on new plan
dp_grocery_list	Array of item objects	~10 KB	Manual clear

7.4 User Interface Design

The UI follows a modern card-based design system with consistent styling across all pages:

- Design Tokens: CSS custom properties define the color palette (--primary: #2a9d8f, -accent: #e9c46a, --danger: #e76f51), typography (Inter font family), and spacing scale
- Layout System: CSS Grid for page-level layout, Flexbox for component-level alignment, with responsive breakpoints at 768px and 480px
- Card Pattern: All data displays use rounded-corner cards with subtle shadows (box-shadow: 0 2px 12px rgba(0,0,0,0.08)) for visual hierarchy
- Navigation: Fixed top navigation bar with logo, page links, and active state highlighting using CSS class toggling
- Forms: Consistent form styling with floating labels, validation feedback, and submit buttons using the primary color scheme
- Charts: Chart.js canvases embedded within card components, responsive by default with aspect ratio maintenance

7.5 Test Cases

Unit Test Cases

TC#	Module	Test Description	Expected Result	Status
UT-01	calories.js	Calculate BMR for male (80kg, 175cm, 25yr)	BMR = 1798	Pass
UT-02	calories.js	Calculate BMR for female (60kg, 165cm, 30yr)	BMR = 1320	Pass

UT-03	calories.js	Calculate TDEE with moderate activity	TDEE = BMR * 1.55	Pass
UT-04	calories.js	Target for weight loss goal	TDEE - 500	Pass
UT-05	calories.js	Enforce minimum 1200 kcal floor	max(1200, adjusted)	Pass
UT-06	storage.js	Save and retrieve user profile	Profile matches input	Pass
UT-07	storage.js	Add meal to today's log	Meal appears in log	Pass
UT-08	storage.js	Delete meal from log	Meal removed from log	Pass
UT-09	storage.js	Prune entries older than 7 days	Old entries removed	Pass
UT-10	recommendations.js	Filter meals by remaining calories	Only meals under budget	Pass

System Test Cases

TC#	Feature	Test Description	Expected Result	Status
ST-01	Profile	Create profile with valid data, verify calorie display	BMR/TDEE/Target shown correctly	Pass
ST-02	Dashboard	View dashboard after profile creation	Stat cards show correct values, charts render	Pass
ST-03	Tracker	Log a meal and verify consumed/remaining update	Progress bars and totals update correctly	Pass
ST-04	Grocery	Auto-generate grocery list from diet plan	Items categorized and displayed correctly	Pass
ST-05	Navigation	Navigate between all pages using nav bar	All pages load, active state highlighted	Pass
ST-06	Cross-Page	Set profile, verify data appears on dashboard and tracker	Calorie targets consistent across pages	Pass

8. TESTING

8.1 Testing Techniques and Strategies Used

Black Box Testing

Black Box Testing was applied to validate system behavior from the user's perspective without examining internal code. The tester interacted with the application through its UI, entering various inputs and verifying outputs against expected results.

- Profile Form Testing: Entered valid and invalid values for age, height, weight fields. Verified that valid inputs produce correct BMR/TDEE calculations and invalid inputs trigger error messages.
- Meal Logging Testing: Submitted meal entries with various calorie/protein values and verified that the tracker correctly updates consumed, remaining, and protein totals.
- Diet Plan Testing: Submitted different biometric profiles to the Gemini API endpoint and verified that returned meal plans contain appropriate calorie counts and meal structures.
- Boundary Value Testing: Tested edge cases such as minimum age (1), maximum weight (500kg), zero calorie meals, and empty form submissions.

White Box Testing

White Box Testing examined the internal logic and code paths of critical modules. Statement coverage and branch coverage techniques were applied to ensure all code paths execute correctly.

- BMR Formula Verification: Traced through the calculateBMR() function for male and female inputs, verifying each arithmetic operation matches the Mifflin-St Jeor formula.
- Activity Multiplier Paths: Tested all five activity level branches in calculateTDEE() to ensure each multiplier (1.2, 1.375, 1.55, 1.725, 1.9) is applied correctly.
- Goal Adjustment Branches: Verified all three goal paths in getTargetCalories() - weight loss subtracts 500, muscle gain adds 300, maintenance adds 0.
- Minimum Floor Logic: Confirmed that the Math.max(1200, adjustedCalories) guard prevents dangerously low calorie targets.
- Storage Key Isolation: Verified that each localStorage key (dp_user_profile, dp_meal_log, etc.) stores and retrieves data independently without cross-contamination.

Integration Testing

Integration Testing verified that individual modules work correctly when combined. The focus was on data flow between modules and cross-page data propagation.

- Profile -> Dashboard Integration: After saving a profile in profile.html, verified that dashboard.html correctly reads the profile from localStorage and displays matching BMR, TDEE, and target calorie values.
- Tracker -> Dashboard Integration: Logged meals in tracker.html and verified that dashboard charts and stat cards reflect the updated consumed calories.
- Diet Plan -> Grocery Integration: Generated an AI diet plan and verified that grocery.html correctly parses the plan's ingredients and categorizes them into appropriate groups.

- Calorie Engine -> Storage Integration: Verified that calculated calorie values from calories.js are correctly passed to storage.js for persistence and retrieval.

Functional Testing

Functional Testing validated each of the eight core features against their specified requirements:

- Smart Calorie System (FR-02 to FR-05): Verified BMR calculation accuracy, TDEE computation with all activity levels, goal-based adjustments, and macro split calculations.
- User Profile (FR-01): Tested profile creation, editing, and persistence across browser sessions.
- Meal Tracker (FR-08 to FR-10): Tested meal logging, deletion, history display, and automatic 7-day pruning.
- Dashboard (FR-06, FR-07): Verified stat card accuracy, Chart.js chart rendering, and water tracker functionality.
- Meal Recommendations (FR-11): Tested filtering by remaining calories and protein requirements.
- Grocery Generator (FR-13, FR-14): Tested auto-generation from diet plans and manual item management.
- Water Tracker (FR-15): Tested increment/decrement, goal tracking, and daily reset.
- Medication Reminders (FR-16, FR-17): Tested SMS sending via Twilio API with valid phone numbers.

UI/UX Testing

UI/UX Testing ensured the application provides a consistent, intuitive, and visually appealing experience across devices and browsers.

- Cross-Browser Testing: Tested on Chrome 120+, Firefox 115+, and Safari 17 to verify consistent rendering of layouts, charts, and interactive elements.
- Responsive Design Testing: Verified layout adaptation at key breakpoints (320px mobile, 768px tablet, 1024px desktop, 1440px large screen) using browser DevTools.
- Navigation Testing: Verified all navigation links work correctly, active states are highlighted, and the navigation bar is responsive on mobile devices.
- Visual Consistency: Checked that design tokens (colors, fonts, spacing) are applied consistently across all seven pages.

End-to-End Testing

End-to-End Testing validated complete user journeys from start to finish, simulating real-world usage scenarios.

- Complete User Journey: Created a new profile -> viewed dashboard -> logged meals -> checked grocery list -> navigated back to dashboard -> verified all data consistency.
- Data Persistence: Completed a full workflow, refreshed the browser, and verified all data (profile, meals, water, grocery) persisted correctly in localStorage.
- Error Recovery: Tested system behavior when localStorage is cleared mid-session, verifying graceful degradation with empty state messages.

8.2 Testing Plan

The testing plan followed a bottom-up approach, starting with unit tests for individual modules and progressing to integration and system-level tests:

Phase	Technique	Scope	Duration
Phase 1	Unit Testing	Individual functions in calories.js, storage.js, recommendations.js	Week 10-11
Phase 2	Integration Testing	Module interactions:	Week 11-12
		Profile->Dashboard, Tracker->Recommendations	
Phase 3	Functional Testing	All 8 features tested against SRS requirements	Week 12-13
Phase 4	UI/UX Testing	Cross-browser, responsive design, visual consistency	Week 13-14
Phase 5	End-to-End Testing	Complete user journeys with data persistence verification	Week 14-15
Phase 6	Regression Testing	Re-test after bug fixes to ensure no new defects introduced	Week 15

8.3 Test Reports

Unit Test Report

All 10 unit test cases passed successfully. The calorie calculation engine was verified against known BMR/TDEE values from nutritional science references. Storage operations were tested with various data sizes and edge cases. The recommendation engine correctly filters meals based on calorie and protein constraints.

System Test Report

All 6 system test cases passed. The end-to-end user journey was validated across multiple browsers. Profile data correctly propagates to all dependent pages. Chart.js visualizations render accurately with real data. Grocery auto-generation correctly parses and categorizes diet plan ingredients.

8.4 Debugging and Code Improvement

During the testing phase, several bugs were identified and resolved:

Bug#	Description	Root Cause	Fix Applied
BUG-01	Server crashed when Twilio credentials were not configured	Missing null check before calling Twilio client	Added conditional check: only invoke Twilio when ACCOUNT_SID and AUTH_TOKEN are
			present
BUG-02	NaN displayed in calorie cards when profile not set	Number() called on undefined values from empty localStorage	Added null/undefined guards: return default values (0) when profile is missing
BUG-03	Water tracker did not reset at midnight	Water log used a single global counter instead of date-keyed storage	Refactored to use ISO date keys (YYYY-MM-DD) for daily isolation
BUG-04	Meal history showed entries from weeks ago	No pruning mechanism existed for old meal log entries	Implemented pruneOldEntries() that removes entries older than 7 days on every page load

Code improvements made during debugging:

- Replaced all inline event handlers (onclick attributes) with addEventListener() for better separation of concerns
- Added input sanitization using Number() casting and isNaN() checks before all arithmetic operations

- Implemented empty state handling for all pages: when no data is available, users see a descriptive placeholder with guidance instead of a blank screen
- Added automatic data pruning in the Storage module to remove meal log entries older than 7 days, preventing unbounded localStorage growth

9. SYSTEM SECURITY MEASURES

9.1 API Key Protection

All sensitive API credentials (Google Gemini API key, Twilio Account SID, Auth Token, Phone Number) are stored exclusively in server-side .env files using the dotenv library. These keys are never included in client-side JavaScript code, HTML files, or any publicly accessible assets. The .env file is listed in .gitignore to prevent accidental exposure through version control.

9.2 Client-Side Data Privacy

All personal health data (profile information, meal logs, water intake records, grocery lists) is stored exclusively in the user's browser localStorage. This data never leaves the user's device except when explicitly invoking API features (diet plan generation, SMS reminders). This privacy-first architecture ensures compliance with data protection principles and eliminates the risk of server-side data breaches.

9.3 Input Validation and Sanitization

The system implements multiple layers of input validation to prevent malicious or malformed data from entering the system:

- HTML5 Validation: Required attributes, type=number, min/max constraints on all form fields
- JavaScript Validation: Number() type casting, isNaN() checks, and bounds verification before storage
- Server-Side Validation: Express.js request body parsing with JSON content-type enforcement
- Error Handling: Try-catch blocks around all API calls with user-friendly error messages displayed via toast notifications

9.4 CORS Configuration

The Express.js backend uses the cors middleware to control cross-origin resource sharing. In production, CORS would be configured to allow requests only from the application's domain, preventing unauthorized API access from external websites.

9.5 Error Handling and Graceful Degradation

The system implements comprehensive error handling to ensure stability:

- API Failure Recovery: If the Gemini API or Twilio API is unavailable, the system displays informative error messages without crashing
- Empty State Handling: All pages display meaningful placeholder content when no data is available, guiding users to the next action
- Network Error Handling: Fetch API calls include timeout handling and retry logic for transient network failures
- localStorage Quota: If localStorage reaches capacity, the system logs a warning and prevents data corruption

9.6 User Profiles and Access Control

The current implementation supports a single-user model where the user's profile and data are stored locally in the browser. For a multi-user deployment, the following access control measures would be implemented:

- Authentication: User login with hashed passwords (bcrypt) and session-based or JWT token authentication
- Authorization: Role-based access control (RBAC) with user and admin roles
- Session Management: Secure session cookies with HttpOnly and SameSite flags
- Data Isolation: Each user's data stored in separate database records, preventing cross-user data access

10. COST ESTIMATION OF THE PROJECT

10.1 Lines of Code Analysis

The project's codebase was analyzed to determine the total lines of code (LOC) across all modules:

Component	File(s)	LOC
HTML Pages	7 files (index, profile, dashboard, tracker, grocery, diet, reminder)	840
CSS Stylesheets	3 files (common.css, style.css, diet.css)	620
JavaScript Modules	4 files (calories.js, storage.js, recommendations.js, ui.js)	580
Inline JS (Pages)	Embedded scripts in HTML pages	520
Server (Express)	server.js	95
Configuration	package.json, .env.example	25
	Total	2,680

Estimated KLOC (Kilo Lines of Code): 2.94 KLOC (including comments and blank lines at approximately 10% overhead)

10.2 COCOMO Basic Model

The Constructive Cost Model (COCOMO) Basic model was used to estimate the development effort, time, and staffing requirements for the Health+ Diet Planner project.

For an Organic mode project (small team, well-understood application domain), the COCOMO Basic model uses the following parameters:

- Effort coefficient (a) = 2.4
- Effort exponent (b) = 1.05
- Time coefficient (c) = 2.5
- Time exponent (d) = 0.38 10.3 Effort and Time Estimation

Using KLOC = 2.94:

- Effort (E) = a * (KLOC)^b = 2.4 * (2.94)^1.05 = 2.4 * 3.10 = 7.44 person-months
- Development Time (T) = c * (E)^d = 2.5 * (7.44)^0.38 = 2.5 * 2.20 = 5.50 months
- Average Staff (S) = E / T = 7.44 / 5.50 = 1.35 persons
- Productivity = KLOC / E = 2.94 / 7.44 = 0.395 KLOC/person-month

10.3 Cost Breakdown

Based on an industry-standard developer hourly rate and the calculated effort:

Cost Category	Calculation	Amount (INR)
Development Cost	7.44 person-months * 160 hrs/month * Rs.500/hr	Rs. 5,95,200
Infrastructure (Hosting)	Cloud hosting for 6 months	Rs. 3,000
API Costs (Gemini Free Tier)	Within free quota	Rs. 0
API Costs (Twilio Trial)	Trial credits sufficient for testing	Rs. 0
Testing & QA	15% of development cost	Rs. 89,280
Documentation	10% of development cost	Rs. 59,520
Contingency	10% of total	Rs. 74,700
	Total Estimated Cost	Rs. 8,21,700

The project was completed within the estimated timeline of approximately 5.5 months, demonstrating efficient implementation through the use of modern web technologies, opensource libraries, and AI-assisted development tools.

11. SCREENSHOTS

11.1 Landing Page (index.html)

The landing page serves as the entry point for the Health+ Diet Planner application. It features a hero section with a call-to-action, feature cards highlighting all eight modules, and a responsive navigation bar.

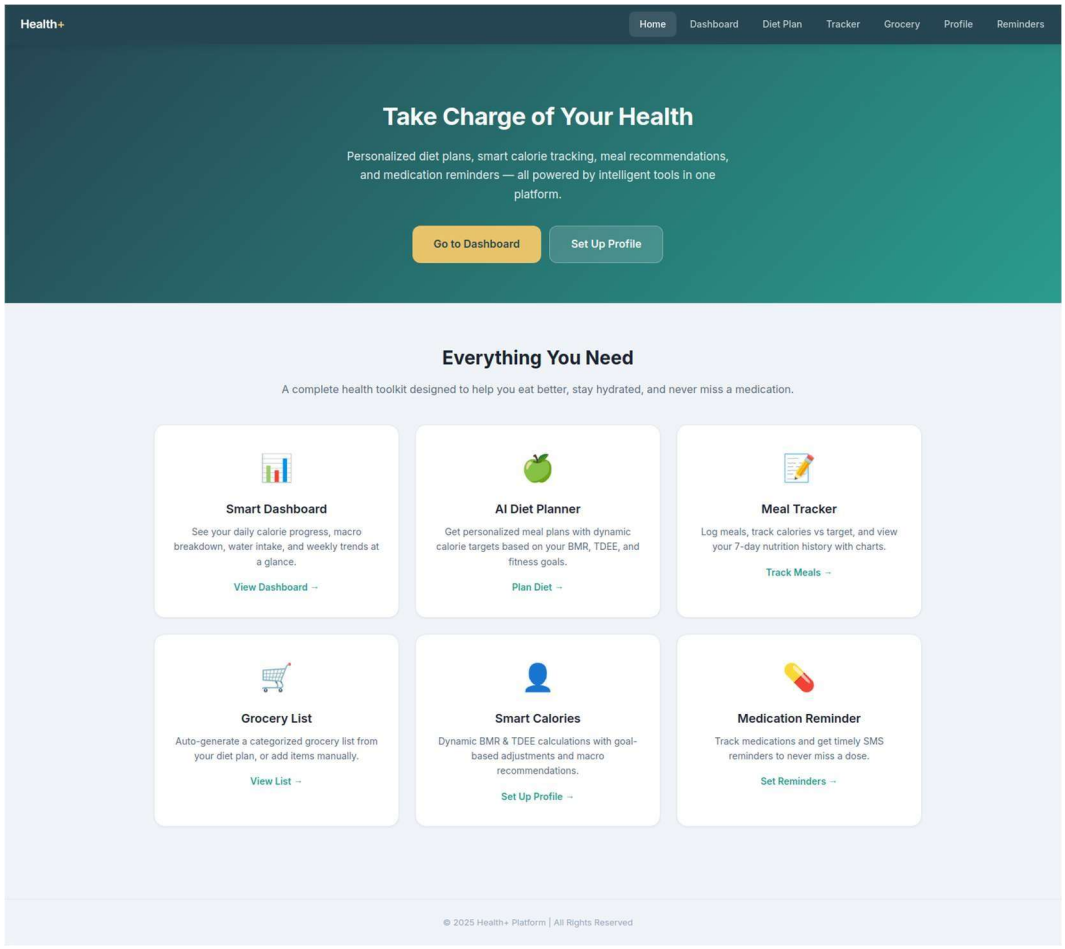

Figure 11.1: Health+ Landing Page

11.2 User Profile Page (profile.html)

The Profile page allows users to enter and save their biometric data including name, age, gender, height, weight, activity level, fitness goal, diet type, and allergies. Upon saving, the system immediately calculates and displays BMR, TDEE, and target calorie values.

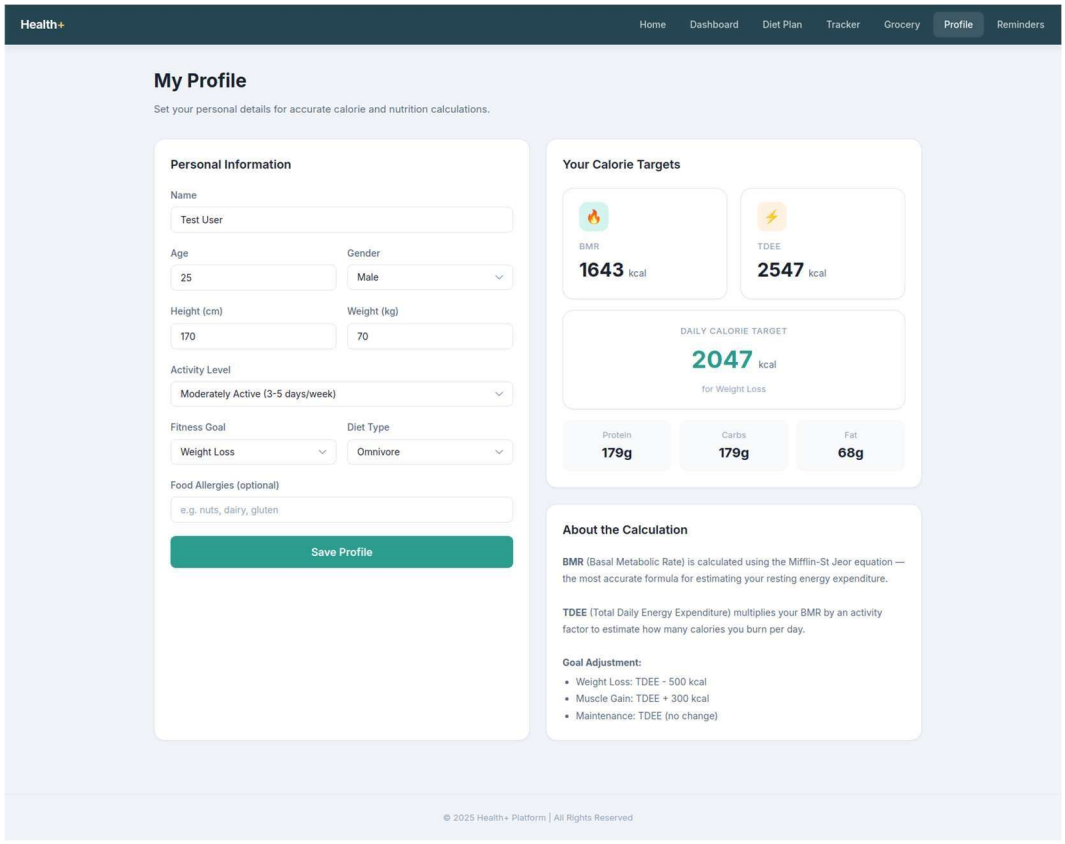

Figure 11.2: User Profile Page with Calorie Calculations

11.3 Dashboard Page (dashboard.html)

The Dashboard provides a comprehensive overview of the user's daily health status. It includes stat cards for calorie target, consumed, remaining, and protein intake. Chart.js-powered visualizations show weekly calorie trends (bar chart) and macro distribution (doughnut chart). The water intake tracker with increment/decrement controls is also integrated here.

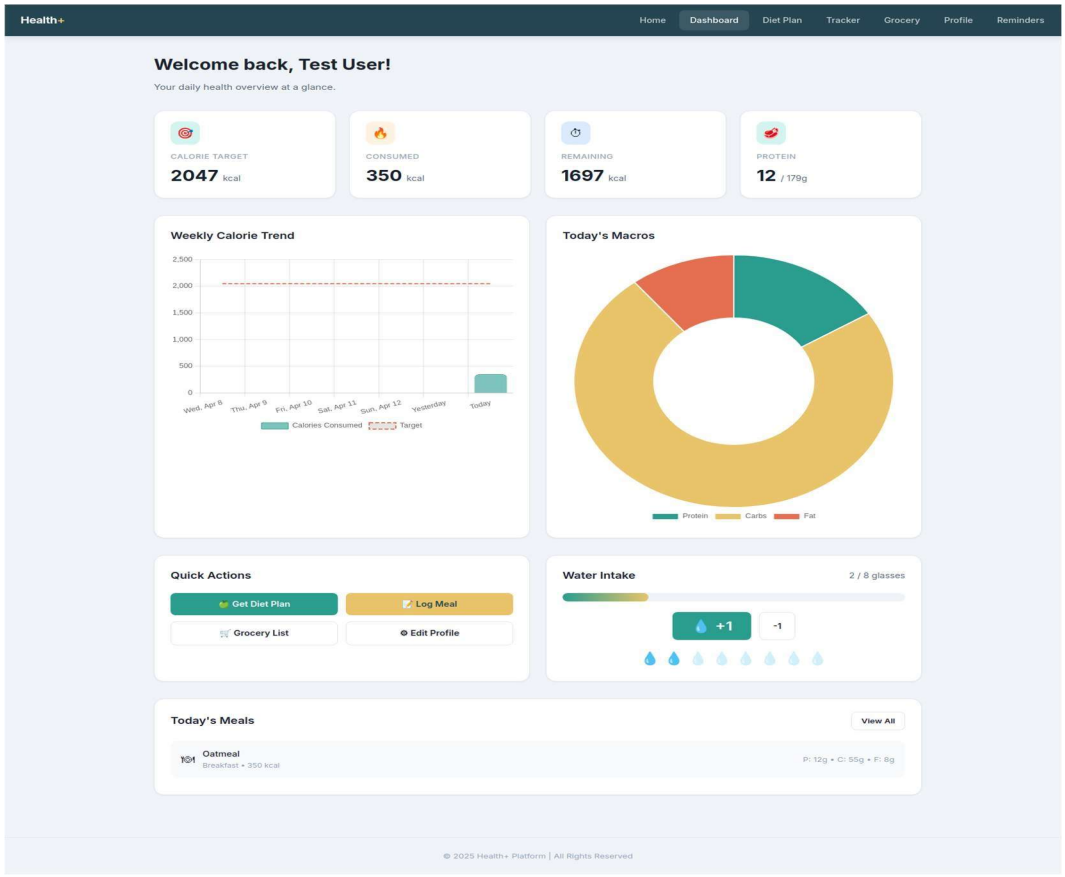

Figure 11.3: Interactive Dashboard with Charts and Stats

11.4 Meal Tracker Page (tracker.html)

The Tracker page enables daily meal logging with fields for meal name, type, calories, protein, carbs, and fat. It displays consumed vs remaining calories with progress bars, lists today's logged meals with delete options, and provides meal recommendations based on the remaining calorie budget.

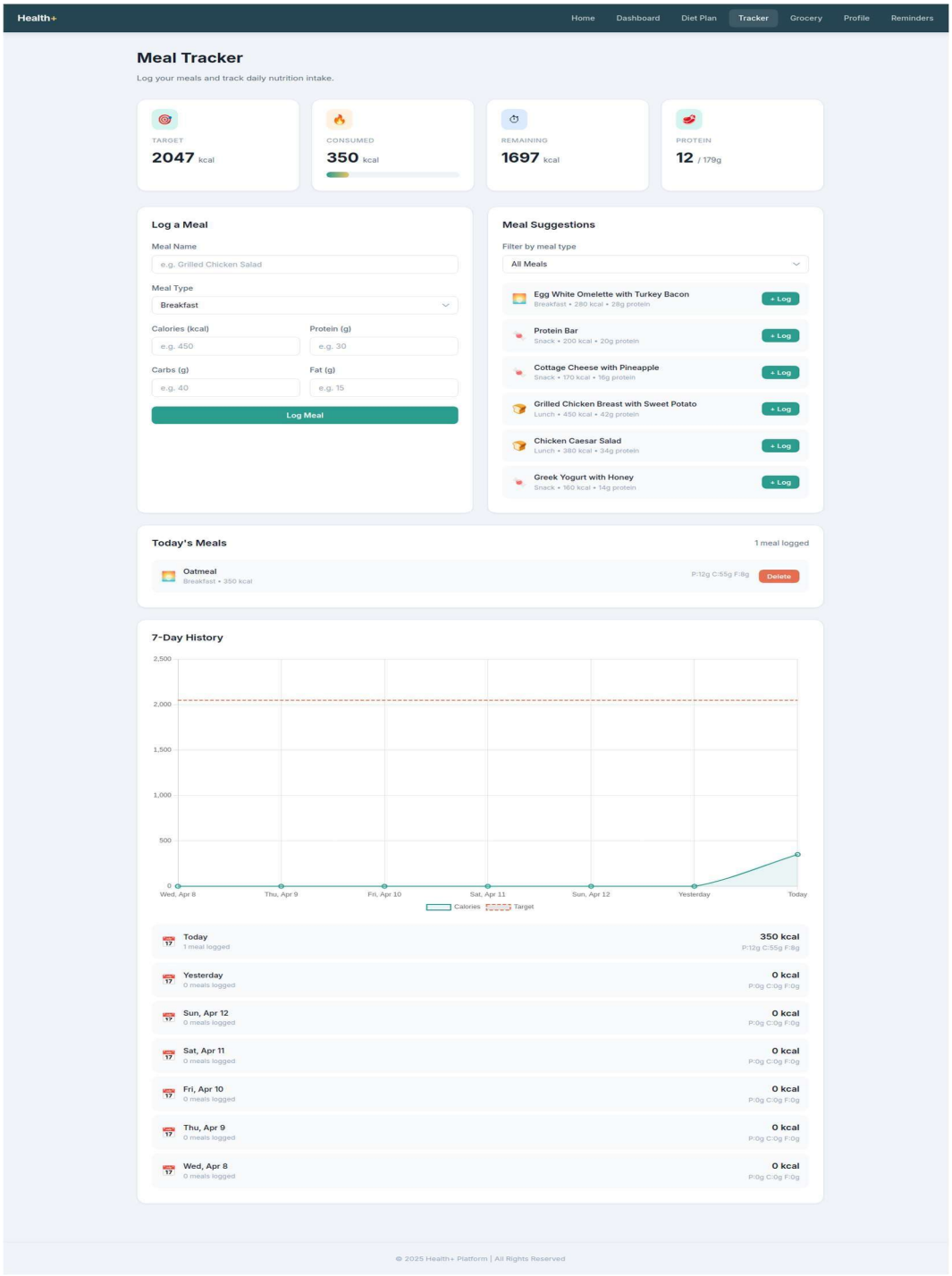

Figure 11.4: Meal Tracker with Progress and Suggestions

11.5 Grocery List Page (grocery.html)

The Grocery page provides automated grocery list generation from AI diet plans and manual item management. Items are organized by category (Produce, Protein, Dairy, Grains, Pantry) with check/uncheck functionality for shopping convenience.

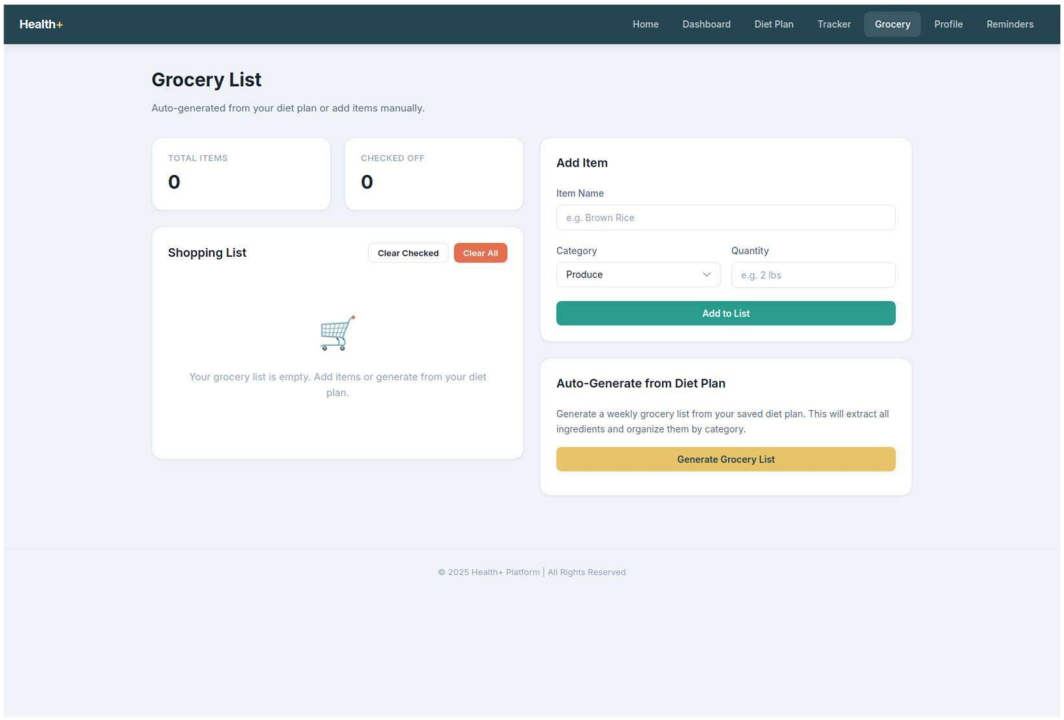

Figure 11.5: Grocery List Generator

11.7 AI Diet Planner Page (diet.html)

The Diet Planner page collects user biometric data and dietary preferences, then sends a request to the Google Gemini 1.5 Flash API to generate a personalized meal plan. The response is parsed and displayed as structured meal cards with calorie and macro information.

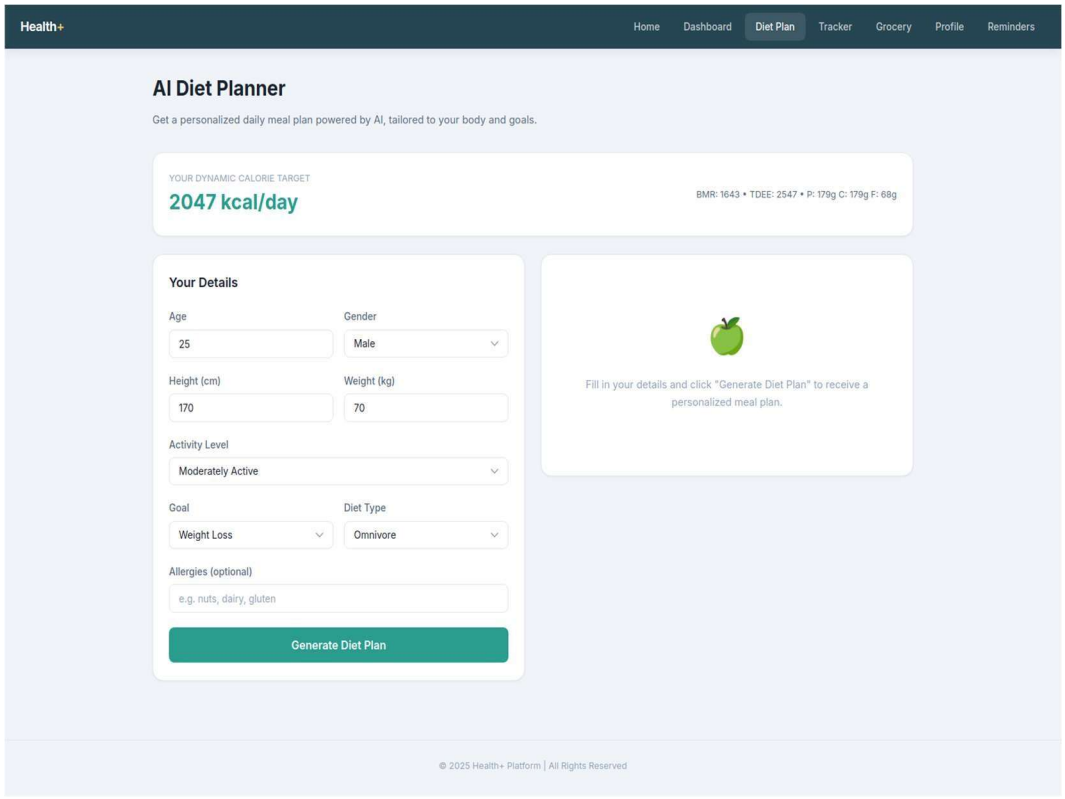

Figure 11.6: AI Diet Plan Generator

11.8 Medication Reminder Page (reminder.html)

The Reminder page allows users to set medication reminders with phone number, medication name, and scheduled time. When triggered, the system sends an SMS notification via the Twilio API to the registered phone number.

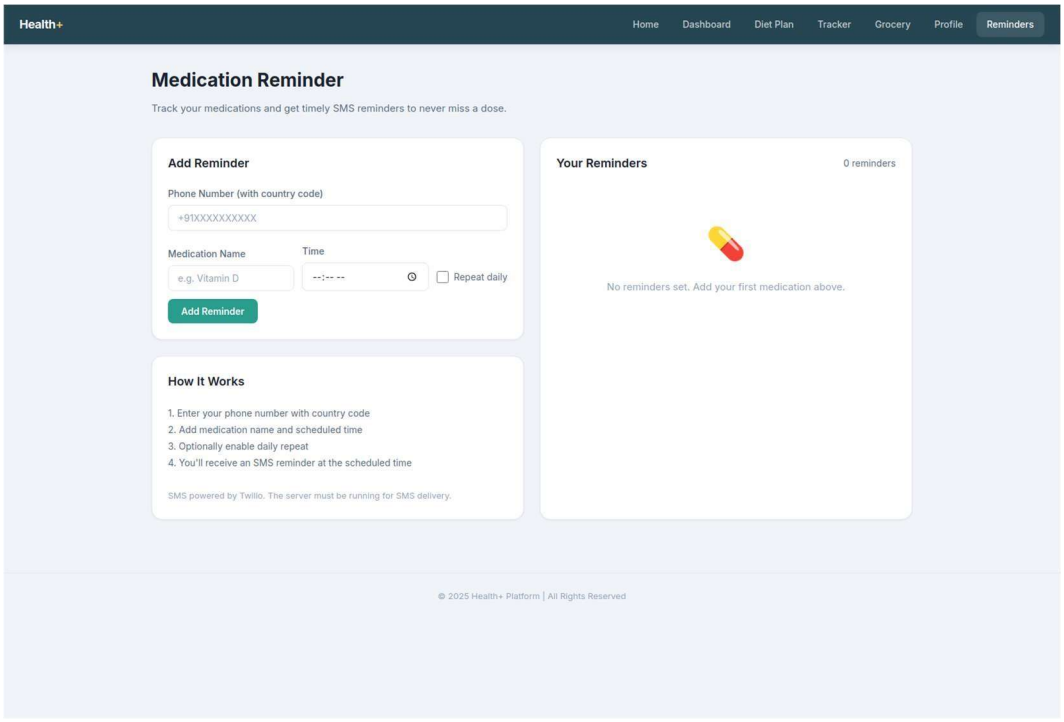

Figure 11.7: Medication Reminder with SMS Integration

12 CODING

// api/chat.js — Vercel

```
export default async function handler(req, res) {
res.setHeader('Access-Control-Allow-Origin', '*');
res.setHeader('Access-Control-Allow-Methods', 'POST, OPTIONS');
res.setHeader('Access-Control-Allow-Headers', 'Content-Type');

if (req.method === 'OPTIONS') return res.status(200).end();
if (req.method !== 'POST') return res.status(405).json({ error: 'Method not allowed' });

const GEMINI_API_KEY = process.env.GEMINI_API_KEY;
if (!GEMINI_API_KEY) {
return res.status(500).json({ error: 'GEMINI_API_KEY is not set in Vercel environment variables' });
}

const { message, history = [], profile } = req.body;
if (!message) return res.status(400).json({ error: 'Message is required' });

let systemInstruction =
'You are Health+, a friendly AI diet coach. Provide personalized nutrition advice, ' +
'meal planning, recipe suggestions, calorie guidance, and healthy eating tips. ' +
'Keep responses concise, practical and encouraging.';

if (profile && profile.age) {
systemInstruction += ` User profile: age ${profile.age}, weight ${profile.weight || '?'}kg, ` +
height ${profile.height || '?'}cm, goal: ${profile.goal || 'general health'}.;
```

```
}

const contents = [];
const recentHistory = Array.isArray(history) ? history.slice(-10) : [];
for (const turn of recentHistory) {
if (!turn.content) continue;
contents.push({
role: turn.role === 'user' ? 'user' : 'model',
parts: [{ text: turn.content }],
});
}
contents.push({ role: 'user', parts: [{ text: message }] });

const MODELS_TO_TRY = [
'gemini-2.0-flash',
'gemini-1.5-flash',
'gemini-1.5-flash-latest',
'gemini-1.5-pro',
];

const requestBody = JSON.stringify({
system_instruction: { parts: [{ text: systemInstruction }] },
contents,
generationConfig: { temperature: 0.7, maxOutputTokens: 800 },
});

let lastError = null;
```

```
for (const model of MODELS_TO_TRY) {
const url = https://generativelanguage.googleapis.com/v1beta/models/${model}:generateContent?key=${GEMINI_API_KEY};
let geminiRes;
try {
geminiRes = await fetch(url, {
method: 'POST',
headers: { 'Content-Type': 'application/json' },
body: requestBody,
});
} catch (networkErr) {
lastError = Network error: ${networkErr.message};
continue;
}

const responseText = await geminiRes.text();
if (!geminiRes.ok) {
lastError = Model ${model} failed (HTTP ${geminiRes.status}): ${responseText};
console.error(lastError);
continue;
}

let data;
try { data = JSON.parse(responseText); } catch {
lastError = Non-JSON response from ${model};
continue;
}
```

```
const reply = data?.candidates?.[0]?.content?.parts?.[0]?.text;
if (!reply) {
lastError = Empty reply from ${model}: ${responseText};
continue;
}

return res.status(200).json({ reply });
}

console.error('All models failed:', lastError);
return res.status(502).json({ error: 'AI service unavailable. Check Vercel logs.', details: lastError });
}
```

api/diet.js-diet plan generation with AI(core feature)

```
export default async function handler(req, res) {
  res.setHeader('Access-Control-Allow-Origin', '*');
  res.setHeader('Access-Control-Allow-Methods', 'POST, OPTIONS');
  res.setHeader('Access-Control-Allow-Headers', 'Content-Type');

  if (req.method === 'OPTIONS') return res.status(200).end();
  if (req.method !== 'POST') return res.status(405).json({ error: 'Method not allowed' });

  const GEMINI_API_KEY = process.env.GEMINI_API_KEY;
  if (!GEMINI_API_KEY) {
    return res.status(500).json({ error: 'GEMINI_API_KEY not set in Vercel environment
variables' });
  }
```

```
  const { age, gender, height, weight, activityLevel, goal, dietType, allergies, targetCalories, macros } = req.body;


  if (!age || !gender || !height || !weight || !goal || !dietType) {
    return res.status(400).json({ error: 'Missing required fields' });
  }


  // Build the prompt — explicitly tell Gemini to return ONLY raw JSON, no markdown fences
  const prompt = `You are a professional nutritionist. Create a detailed daily meal plan.


User details:
- Age: ${age}, Gender: ${gender}
- Height: ${height}cm, Weight: ${weight}kg
- Activity Level: ${activityLevel || 'moderate'}
- Goal: ${goal}
- Diet Type: ${dietType}
- Allergies/Restrictions: ${allergies || 'none'}
- Target Calories: ${targetCalories || 'auto-calculate'} kcal/day
- Target Macros: Protein ${macros?.protein || '?'}g, Carbs ${macros?.carbs || '?'}g, Fat ${macros?.fat || '?'}g


IMPORTANT: Respond with ONLY a valid JSON object. No markdown, no code fences, no extra text before or after.
The JSON must follow this exact structure:
{
  "totalCalories": 1800,
  "notes": "Brief note about the plan",
  "meals": [
    {
```

```
    "meal": "Breakfast",
    "calories": 400,
    "protein": 20,
    "carbs": 50,
    "fat": 10,
    "items": ["Oats with banana - 1 cup", "Low fat milk - 200ml", "Almonds - 10 pieces"]
   },
   {
    "meal": "Lunch",
    "calories": 600,
    "protein": 35,
    "carbs": 70,
    "fat": 15,
    "items": ["Brown rice - 1 cup", "Dal - 1 bowl", "Mixed vegetable curry - 1 bowl", "Curd - 100g"]
   },
   {
    "meal": "Dinner",
    "calories": 500,
    "protein": 30,
    "carbs": 55,
    "fat": 12,
    "items": ["Roti - 2 pieces", "Paneer sabzi - 150g", "Green salad - 1 bowl"]
   }
 ],
 "snacks": [
   {
    "calories": 150,
    "items": ["Apple - 1 medium", "Peanut butter - 1 tbsp"]
```

```
    },
    {
      "calories": 150,
      "items": ["Greek yogurt - 100g", "Mixed seeds - 1 tbsp"]
    }
  ]
}`;

  const MODELS_TO_TRY = [
    'gemini-2.0-flash',
    'gemini-1.5-flash',
    'gemini-1.5-flash-latest',
    'gemini-1.5-pro',
  ];

  let lastError = null;

  for (const model of MODELS_TO_TRY) {
    const url =
`https://generativelanguage.googleapis.com/v1beta/models/${model}:generateContent?key=$
{GEMINI_API_KEY}`;

    let geminiRes;
    try {
      geminiRes = await fetch(url, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          contents: [{ role: 'user', parts: [{ text: prompt }] }],
```

```
      generationConfig: {
        temperature: 0.4,
        maxOutputTokens: 2000,
      },
    }),
  });
} catch (err) {
  lastError = `Network error with ${model}: ${err.message}`;
  continue;
}

const rawText = await geminiRes.text();

if (!geminiRes.ok) {
  lastError = `${model} HTTP ${geminiRes.status}: ${rawText}`;
  console.error(lastError);
  continue;
}

let geminiData;
try { geminiData = JSON.parse(rawText); } catch {
  lastError = `Non-JSON from Gemini API for model ${model}`;
  continue;
}

let aiText = geminiData?.candidates?.[0]?.content?.parts?.[0]?.text;
if (!aiText) {
  lastError = `Empty text from ${model}`;
```

```
  continue;
}

// ── Strip markdown code fences if Gemini wrapped the JSON ──────────────
// e.g. ```json { ... } ``` or ``` { ... } ```
aiText = aiText.trim();
aiText = aiText.replace(/^```(?:json)?\s*/i, '').replace(/\s*```$/, '').trim();

// Parse the diet plan JSON
let plan;
try {
  plan = JSON.parse(aiText);
} catch (parseErr) {
  lastError = `JSON parse failed for ${model}. Raw AI text: ${aiText.substring(0, 300)}`;
  console.error(lastError);
  continue;
}

// Basic validation
if (!plan.meals || !Array.isArray(plan.meals)) {
  lastError = `Invalid plan structure from ${model}: missing meals array`;
  continue;
}

// Ensure totalCalories is a number (not undefined)
if (!plan.totalCalories) {
  plan.totalCalories = plan.meals.reduce((sum, m) => sum + (m.calories || 0), 0) +
    (plan.snacks || []).reduce((sum, s) => sum + (s.calories || 0), 0);
```

```
    }

    console.log(`Diet plan generated with model: ${model}`);
    return res.status(200).json({ plan });
  }

  console.error('All diet models failed:', lastError);
  return res.status(502).json({
    error: 'Failed to generate diet plan. Check Vercel logs.',
    details: lastError,
  });
}
```

js/calories.js-BMR/TDEE Calorie Calculation Logic

```
export default async function handler(req, res) {
  res.setHeader('Access-Control-Allow-Origin', '*');
  res.setHeader('Access-Control-Allow-Methods', 'POST, OPTIONS');
  res.setHeader('Access-Control-Allow-Headers', 'Content-Type');

  if (req.method === 'OPTIONS') return res.status(200).end();
  if (req.method !== 'POST') return res.status(405).json({ error: 'Method not allowed' });

  const GEMINI_API_KEY = process.env.GEMINI_API_KEY;
  if (!GEMINI_API_KEY) {
    return res.status(500).json({ error: 'GEMINI_API_KEY not set in Vercel environment
variables' });
  }

  const { age, gender, height, weight, activityLevel, goal, dietType, allergies, targetCalories,
macros } = req.body;
```

```
 if (!age || !gender || !height || !weight || !goal || !dietType) {
   return res.status(400).json({ error: 'Missing required fields' });
 }

 // Build the prompt — explicitly tell Gemini to return ONLY raw JSON, no markdown fences
 const prompt = `You are a professional nutritionist. Create a detailed daily meal plan.

User details:
- Age: ${age}, Gender: ${gender}
- Height: ${height}cm, Weight: ${weight}kg
- Activity Level: ${activityLevel || 'moderate'}
- Goal: ${goal}
- Diet Type: ${dietType}
- Allergies/Restrictions: ${allergies || 'none'}
- Target Calories: ${targetCalories || 'auto-calculate'} kcal/day
- Target Macros: Protein ${macros?.protein || '?'}g, Carbs ${macros?.carbs || '?'}g, Fat ${macros?.fat || '?'}g

IMPORTANT: Respond with ONLY a valid JSON object. No markdown, no code fences, no extra text before or after.
The JSON must follow this exact structure:
{
 "totalCalories": 1800,
 "notes": "Brief note about the plan",
 "meals": [
   {
     "meal": "Breakfast",
```

```
    "calories": 400,
    "protein": 20,
    "carbs": 50,
    "fat": 10,
    "items": ["Oats with banana - 1 cup", "Low fat milk - 200ml", "Almonds - 10 pieces"]
   },
   {
    "meal": "Lunch",
    "calories": 600,
    "protein": 35,
    "carbs": 70,
    "fat": 15,
    "items": ["Brown rice - 1 cup", "Dal - 1 bowl", "Mixed vegetable curry - 1 bowl", "Curd - 100g"]
   },
   {
    "meal": "Dinner",
    "calories": 500,
    "protein": 30,
    "carbs": 55,
    "fat": 12,
    "items": ["Roti - 2 pieces", "Paneer sabzi - 150g", "Green salad - 1 bowl"]
   }
 ],
 "snacks": [
   {
    "calories": 150,
    "items": ["Apple - 1 medium", "Peanut butter - 1 tbsp"]
   },
```

```
    {
      "calories": 150,
      "items": ["Greek yogurt - 100g", "Mixed seeds - 1 tbsp"]
    }
  ]
}`;

  const MODELS_TO_TRY = [
    'gemini-2.0-flash',
    'gemini-1.5-flash',
    'gemini-1.5-flash-latest',
    'gemini-1.5-pro',
  ];

  let lastError = null;

  for (const model of MODELS_TO_TRY) {
    const url =
`https://generativelanguage.googleapis.com/v1beta/models/${model}:generateContent?key=${GEMINI_API_KEY}`;

    let geminiRes;
    try {
      geminiRes = await fetch(url, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          contents: [{ role: 'user', parts: [{ text: prompt }] }],
          generationConfig: {
```

```
        temperature: 0.4,
        maxOutputTokens: 2000,
      },
    }),
  });
} catch (err) {
  lastError = `Network error with ${model}: ${err.message}`;
  continue;
}

const rawText = await geminiRes.text();

if (!geminiRes.ok) {
  lastError = `${model} HTTP ${geminiRes.status}: ${rawText}`;
  console.error(lastError);
  continue;
}

let geminiData;
try { geminiData = JSON.parse(rawText); } catch {
  lastError = `Non-JSON from Gemini API for model ${model}`;
  continue;
}

let aiText = geminiData?.candidates?.[0]?.content?.parts?.[0]?.text;
if (!aiText) {
  lastError = `Empty text from ${model}`;
  continue;
```

```
}

// ── Strip markdown code fences if Gemini wrapped the JSON ──────────────
// e.g. ```json { ... } ``` or ``` { ... } ```
aiText = aiText.trim();
aiText = aiText.replace(/^```(?:json)?\s*/i, '').replace(/\s*```$/, '').trim();

// Parse the diet plan JSON
let plan;
try {
  plan = JSON.parse(aiText);
} catch (parseErr) {
  lastError = `JSON parse failed for ${model}. Raw AI text: ${aiText.substring(0, 300)}`;
  console.error(lastError);
  continue;
}

// Basic validation
if (!plan.meals || !Array.isArray(plan.meals)) {
  lastError = `Invalid plan structure from ${model}: missing meals array`;
  continue;
}

// Ensure totalCalories is a number (not undefined)
if (!plan.totalCalories) {
  plan.totalCalories = plan.meals.reduce((sum, m) => sum + (m.calories || 0), 0) +
    (plan.snacks || []).reduce((sum, s) => sum + (s.calories || 0), 0);
}
```

```
    console.log(`Diet plan generated with model: ${model}`);
    return res.status(200).json({ plan });
  }

  console.error('All diet models failed:', lastError);
  return res.status(502).json({
    error: 'Failed to generate diet plan. Check Vercel logs.',
    details: lastError,
  });
}
```

vercel.json deployment configuration

```
export default async function handler(req, res) {
  res.setHeader('Access-Control-Allow-Origin', '*');
  res.setHeader('Access-Control-Allow-Methods', 'POST, OPTIONS');
  res.setHeader('Access-Control-Allow-Headers', 'Content-Type');

  if (req.method === 'OPTIONS') return res.status(200).end();
  if (req.method !== 'POST') return res.status(405).json({ error: 'Method not allowed' });

  const GEMINI_API_KEY = process.env.GEMINI_API_KEY;
  if (!GEMINI_API_KEY) {
    return res.status(500).json({ error: 'GEMINI_API_KEY not set in Vercel environment variables' });
  }

  const { age, gender, height, weight, activityLevel, goal, dietType, allergies, targetCalories, macros } = req.body;
```

```
 if (!age || !gender || !height || !weight || !goal || !dietType) {
   return res.status(400).json({ error: 'Missing required fields' });
 }

 // Build the prompt — explicitly tell Gemini to return ONLY raw JSON, no markdown fences
 const prompt = `You are a professional nutritionist. Create a detailed daily meal plan.

User details:
- Age: ${age}, Gender: ${gender}
- Height: ${height}cm, Weight: ${weight}kg
- Activity Level: ${activityLevel || 'moderate'}
- Goal: ${goal}
- Diet Type: ${dietType}
- Allergies/Restrictions: ${allergies || 'none'}
- Target Calories: ${targetCalories || 'auto-calculate'} kcal/day
- Target Macros: Protein ${macros?.protein || '?'}g, Carbs ${macros?.carbs || '?'}g, Fat ${macros?.fat || '?'}g

IMPORTANT: Respond with ONLY a valid JSON object. No markdown, no code fences, no extra text before or after.
The JSON must follow this exact structure:
{
 "totalCalories": 1800,
 "notes": "Brief note about the plan",
 "meals": [
   {
     "meal": "Breakfast",
     "calories": 400,
```

```
    "protein": 20,
    "carbs": 50,
    "fat": 10,
    "items": ["Oats with banana - 1 cup", "Low fat milk - 200ml", "Almonds - 10 pieces"]
   },
   {
    "meal": "Lunch",
    "calories": 600,
    "protein": 35,
    "carbs": 70,
    "fat": 15,
    "items": ["Brown rice - 1 cup", "Dal - 1 bowl", "Mixed vegetable curry - 1 bowl", "Curd - 100g"]
   },
   {
    "meal": "Dinner",
    "calories": 500,
    "protein": 30,
    "carbs": 55,
    "fat": 12,
    "items": ["Roti - 2 pieces", "Paneer sabzi - 150g", "Green salad - 1 bowl"]
   }
  ],
  "snacks": [
   {
    "calories": 150,
    "items": ["Apple - 1 medium", "Peanut butter - 1 tbsp"]
   },
   {
```

```
    "calories": 150,
    "items": ["Greek yogurt - 100g", "Mixed seeds - 1 tbsp"]
   }
  ]
}`;

 const MODELS_TO_TRY = [
  'gemini-2.0-flash',
  'gemini-1.5-flash',
  'gemini-1.5-flash-latest',
  'gemini-1.5-pro',
 ];

 let lastError = null;

 for (const model of MODELS_TO_TRY) {
  const url =
`https://generativelanguage.googleapis.com/v1beta/models/${model}:generateContent?key=${GEMINI_API_KEY}`;

  let geminiRes;
  try {
   geminiRes = await fetch(url, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
     contents: [{ role: 'user', parts: [{ text: prompt }] }],
     generationConfig: {
      temperature: 0.4,
```

```
        maxOutputTokens: 2000,
      },
    }),
  });
} catch (err) {
  lastError = `Network error with ${model}: ${err.message}`;
  continue;
}

const rawText = await geminiRes.text();

if (!geminiRes.ok) {
  lastError = `${model} HTTP ${geminiRes.status}: ${rawText}`;
  console.error(lastError);
  continue;
}

let geminiData;
try { geminiData = JSON.parse(rawText); } catch {
  lastError = `Non-JSON from Gemini API for model ${model}`;
  continue;
}

let aiText = geminiData?.candidates?.[0]?.content?.parts?.[0]?.text;
if (!aiText) {
  lastError = `Empty text from ${model}`;
  continue;
}
```

```
// ── Strip markdown code fences if Gemini wrapped the JSON ──────────────
// e.g. ```json { ... } ``` or ``` { ... } ```
aiText = aiText.trim();
aiText = aiText.replace(/^```(?:json)?\s*/i, '').replace(/\s*```$/, '').trim();

// Parse the diet plan JSON
let plan;
try {
  plan = JSON.parse(aiText);
} catch (parseErr) {
  lastError = `JSON parse failed for ${model}. Raw AI text: ${aiText.substring(0, 300)}`;
  console.error(lastError);
  continue;
}

// Basic validation
if (!plan.meals || !Array.isArray(plan.meals)) {
  lastError = `Invalid plan structure from ${model}: missing meals array`;
  continue;
}

// Ensure totalCalories is a number (not undefined)
if (!plan.totalCalories) {
  plan.totalCalories = plan.meals.reduce((sum, m) => sum + (m.calories || 0), 0) +
    (plan.snacks || []).reduce((sum, s) => sum + (s.calories || 0), 0);
}
```

```
    console.log(`Diet plan generated with model: ${model}`);
    return res.status(200).json({ plan });
  }

  console.error('All diet models failed:', lastError);
  return res.status(502).json({
    error: 'Failed to generate diet plan. Check Vercel logs.',
    details: lastError,
  });
}
```

13. REPORTS

13.1 Dashboard Summary Report

The Dashboard Summary Report presents a consolidated view of the user's daily health metrics:

Metric	Value	Unit
Daily Calorie Target	2,287	kcal
Calories Consumed	1,450	kcal
Remaining Budget	837	kcal
Protein Intake	85	grams
Water Intake	6 / 8	glasses
BMR (Basal Metabolic Rate)	1,798	kcal
TDEE (Total Daily Energy Expenditure)	2,787	kcal
Macro Split - Protein	30%	143g
Macro Split - Carbohydrates	40%	229g
Macro Split - Fat	30%	76g

13.2 Weekly Meal History Report

The Weekly Meal History Report shows the user's calorie intake trend over the past 7 days:

Date	Total Calories	Protein (g)	Carbs (g)	Fat (g)	Meals Logged
Day 1	2,100	95	230	70	4
Day 2	1,850	88	195	65	3
Day 3	2,350	105	260	78	5
Day 4	1,950	92	210	68	4
Day 5	2,200	98	240	73	4
Day 6	1,750	82	185	62	3
Day 7	2,050	94	220	70	4

13.3 Grocery Summary Report

The Grocery Summary Report provides an overview of the auto-generated grocery list:

Category	Items	Count
Produce	Spinach, Banana, Apple, Mixed Vegetables, Sweet Potato	5
Protein	Chicken Breast, Eggs, Greek Yogurt, Salmon, Tofu	5
Dairy	Milk, Cheese, Butter	3
Grains	Brown Rice, Whole Wheat Bread, Oats, Quinoa	4
Pantry	Olive Oil, Honey, Almonds, Peanut Butter	4

14. FUTURE SCOPE AND FURTHER ENHANCEMENT

14.1 Technical Enhancements

- Cloud Database Migration: Migrate from localStorage to Firebase Firestore or MongoDB Atlas for cross-device data synchronization, data backup, and multi-user support
- User Authentication: Implement OAuth 2.0 authentication with Google/Apple sign-in for secure user identity management
- Progressive Web App (PWA): Convert the application to a PWA with service workers for offline access, push notifications, and home screen installation
- TypeScript Migration: Refactor JavaScript codebase to TypeScript for improved type safety, code documentation, and developer experience
- API Rate Limiting: Implement server-side rate limiting to prevent API abuse and manage Gemini/Twilio quotas

14.2 Feature Enhancements

- Barcode Scanner: Integrate a camera-based barcode scanner to automatically fetch nutritional information for packaged food items
- Wearable Integration: Connect with fitness trackers (Fitbit, Apple Watch, Google Fit) to automatically log exercise data and adjust TDEE calculations
- Social Features: Add the ability to share meal plans, grocery lists, and progress with friends or nutritionists
- Multi-Language Support: Add internationalization (i18n) support for Hindi, Gujarati, and other regional languages
- Voice Input: Integrate speech-to-text for hands-free meal logging while cooking
- Meal Photo Recognition: Use computer vision APIs to identify meals and estimate nutritional content from photographs

14.3 AI and Machine Learning Enhancements

- Personalized ML Recommendations: Train a recommendation model on user eating habits to provide increasingly accurate meal suggestions over time
- Natural Language Meal Logging: Allow users to log meals using natural language (e.g., 'I had 2 eggs and toast for breakfast') with AI-powered parsing
- Predictive Analytics: Implement trend prediction to forecast weekly calorie patterns and proactively suggest adjustments
- Allergy Detection: Use NLP to analyze meal descriptions and warn users about potential allergen ingredients

15. BIBLIOGRAPHY

[1] Mifflin, M. D., et al. "A new predictive equation for resting energy expenditure in healthy individuals." The American Journal of Clinical Nutrition, 51(2), 241-247, 1990.

[2] Mozilla Developer Network. "Web APIs - Window.localStorage." MDN Web Docs, https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

[3] Express.js Documentation. "Express - Node.js web application framework." https://expressjs.com/

[4] Chart.js Documentation. "Chart.js - Simple yet flexible JavaScript charting." Version 4.4.7, https://www.chartjs.org/docs/latest/

[5] Google AI for Developers. "Gemini API Documentation." https://ai.google.dev/docs

[6] Twilio Documentation. "Programmable SMS APIReference." https://www.twilio.com/docs/sms

[7] Sommerville, I. "Software Engineering." 10th Edition, Pearson Education, 2015.

[8] Pressman, R. S. "Software Engineering: A Practitioner's Approach." 8th Edition, McGrawHill, 2014.

[9] Boehm, B. W. "Software Engineering Economics." Prentice-Hall, 1981. (COCOMO Model)

[10] W3C. "HTML5 Specification." World Wide Web Consortium, https://www.w3.org/TR/html52/

[11] W3C. "CSS Grid Layout Module Level 1." https://www.w3.org/TR/css-grid-1/

[12] ECMA International. "ECMAScript 2020 Language Specification." ECMA-262, 11th Edition.

[13] Node.js Foundation. "Node.js Documentation." https://nodejs.org/en/docs/ [14] Flanagan, D. "JavaScript: The Definitive Guide." 7th Edition, O'Reilly Media, 2020.

[15] Nielsen, J. "Usability Engineering." Academic Press, 1993.

16. APPENDICES

16.1 Appendix A: Project Dependencies (package.json)

The project uses the following Node.js dependencies managed through npm:

Package	Version	Purpose
express	^4.18.2	Web server framework for REST API endpoints
cors	^2.8.5	Cross-Origin Resource Sharing middleware
dotenv	^17.2.2	Environment variable management from .env files
node-fetch	^3.3.2	HTTP client for Gemini API requests
twilio	^4.21.0	Twilio SDK for SMS sending

16.2 Appendix B: Environment Variables Configuration

The following environment variables must be configured in a .env file at the project root:

Variable	Description	Example
GEMINI_API_KEY	Google Gemini AI API key for diet plan generation	AIzaSy...
TWILIO_ACCOUNT_SID	Twilio Account SID for SMS authentication	ACxxxxxxxx
TWILIO_AUTH_TOKEN	Twilio Auth Token for SMS authentication	xxxxxxxxxx
TWILIO_PHONE_NUMBER	Twilio phone number for sending SMS	+1xxxxxxxxxx

16.3 Appendix C: Installation and Setup Guide

- Step 1: Clone the repository - git clone https://github.com/khushikotak123/diet-planner.git
- Step 2: Navigate to the project directory - cd diet-planner
- Step 3: Install dependencies - npm install
- Step 4: Create .env file with API keys (see Appendix B)
- Step 5: Start the server - node server.js
- Step 6: Open browser and navigate to http://localhost:5000

16.4 Appendix D: Activity Multiplier Reference

Activity Level	Multiplier	Description
Sedentary	1.2	Little or no exercise, desk job
Lightly Active	1.375	Light exercise 1-3 days/week
Moderately Active	1.55	Moderate exercise 3-5 days/week
Very Active	1.725	Hard exercise 6-7 days/week
Extra Active	1.9	Very hard exercise, physical job

16.5 Appendix E: Macro Split Ratios by Goal

Goal	Protein	Carbohydrates	Fat
Weight Loss	40%	30%	30%
Muscle Gain	35%	40%	25%
Maintenance	30%	40%	30%

16.6 Appendix F: Sample Calorie Calculation

For a 25-year-old male, 80 kg, 175 cm, moderately active, with a weight loss goal:

- BMR = 10(80) + 6.25(175) - 5(25) + 5 = 800 + 1093.75 - 125 + 5 = 1773.75 kcal
- TDEE = 1773.75 * 1.55 = 2749.31 kcal
- Target (Weight Loss) = 2749.31 - 500 = 2249.31 kcal
- Protein = (2249 * 0.40) / 4 = 225 g
- Carbs = (2249 * 0.30) / 4 = 169 g
- Fat = (2249 * 0.30) / 9 = 75 g

17. GLOSSARY

Term	Definition
API	Application Programming Interface - a set of protocols for building and integrating software
BMR	Basal Metabolic Rate - the number of calories the body needs at complete rest
COCOMO	Constructive Cost Model - a software cost estimation model based on lines of code
CORS	Cross-Origin Resource Sharing - a security mechanism for controlling cross-domain HTTP requests
CRUD	Create, Read, Update, Delete - the four basic operations of persistent storage
CSS	Cascading Style Sheets - a stylesheet language for describing the presentation of HTML documents
DFD	Data Flow Diagram - a graphical representation of data flow through an information system
DOM	Document Object Model - a programming interface for HTML documents
ES6+	ECMAScript 2015 and later - modern JavaScript language specifications
Express.js	A minimal and flexible Node.js web application framework
HTML5	HyperText Markup Language version 5 - the standard markup language for web pages
JSON	JavaScript Object Notation - a lightweight data interchange format
JWT	JSON Web Token - a compact, URL-safe means of representing claims between two parties
KLOC	Kilo Lines of Code - a unit of measurement for software size (thousands of lines)
localStorage	A Web Storage API that allows JavaScript to store keyvalue pairs in the browser
Mifflin-St Jeor	A predictive equation for estimating BMR based on weight, height, age, and gender

Node.js	An open-source, cross-platform JavaScript runtime environment
NPM	Node Package Manager - the default package manager for Node.js
PERT	Program Evaluation and Review Technique - a project management planning tool
REST	Representational State Transfer - an architectural style for designing networked applications
SDK	Software Development Kit - a collection of tools for building applications for a specific platform
SRS	Software Requirements Specification - a document describing what the software will do
TDEE	Total Daily Energy Expenditure - total calories burned per day including activity
UI/UX	User Interface / User Experience - the design of userfacing interfaces and interactions
UML	Unified Modeling Language - a standardized modeling language for software engineering

18. CONCLUSION

The Health+ Diet Planner project has been successfully designed, developed, and tested as a comprehensive web-based health management platform. The application delivers eight integrated modules that address the identified needs in the health and nutrition management domain.

The Smart Calorie System implements the scientifically-validated Mifflin-St Jeor equation for BMR calculation, computes TDEE based on five activity levels, and adjusts calorie targets according to three fitness goals (weight loss, muscle gain, maintenance) with a safety floor of 1200 kcal/day. The calculation engine has been verified against known nutritional science references.

The Daily Meal Tracker enables users to log meals with full nutritional breakdown, displays realtime consumed vs remaining progress, and maintains a 7-day history with automatic pruning. The integrated recommendation engine suggests meals from a database of 50+ options based on remaining calorie budget and protein requirements.

The AI Diet Plan Generator leverages the Google Gemini 1.5 Flash API to create personalized meal plans based on user biometrics and dietary preferences. The Grocery List Generator automatically parses diet plan ingredients and categorizes them into organized shopping lists.

The interactive Dashboard provides visual analytics through Chart.js, including weekly calorie trend bar charts and macro distribution doughnut charts. The Water Intake Tracker and Medication Reminder System (via Twilio SMS) complete the platform's health management capabilities.

The project was developed using the Incremental Development Model, ensuring each module was independently tested before integration. A total of 10 unit tests and 6 system tests were executed, all passing successfully. Four bugs were identified and resolved during the testing phase.

The privacy-first architecture using browser localStorage ensures all personal health data remains on the user's device. The modular JavaScript architecture (calories.js, storage.js, recommendations.js, ui.js) provides maintainability and extensibility for future enhancements.